

Módulo: Programación de Servicios y Procesos

UT2: PROGRAMACIÓN MULTIHILO

Código **0490**

Nivel **Ciclo Formativo de Grado Superior**
Ciclo **Desarrollo de Aplicaciones Multiplataforma**

Índice

1 Introducción a la programación multihilo 5
Resultados de aprendizaje y criterios de evaluación 5
Objetivos de la unidad de trabajo 5
Ejemplos de la programación multihilo 7
Desafíos de la programación multihilo 7
Contexto actual de la programación multihilo 8
2 Conceptos fundamentales 10
2.1 Hilos y procesos 10
Hilos frente a procesos 10
2.2 Recursos compartidos por los hilos 11
Recursos que NO comparten los hilos 11
Recursos que SÍ comparten los hilos 11
Riesgos de compartir 12
2.3 Ventajas en el uso de hilos 12
¿Cuándo utilizar hilos en una aplicación? 13
3 Estados de un hilo y su ciclo de vida 14
3.1 Estados principales de un hilo 14
Transición entre estados 14
Método getState() 16
Diferencias entre hilos y procesos 17
Diferencias entre los estados de los hilos y de los procesos 17
3.2 Librerías y clases multihilo en Java 17
3.2.1 Utilidades de concurrencia en java.lang 18
3.2.2 Utilidades de concurrencia en java.util.concurrent 19
Clases de sincronización 19
Interfaces para ejecutar tareas 20
Gestión de colas concurrentes (BlockingQueue) 21
Paquete java.util.concurrent.atomic 21
Paquete java.util.concurrent.locks 21
3.3 Creación y gestión de hilos 23
3.3.1 Creación de hilos en Java 23
Creación de hilos extendiendo la clase Thread 23

Creación de hilos implementando la interfaz Runnable 24
Elección entre Thread y Runnable 25
3.3.2 Gestión y planificación de hilos 25
Tipos de ejecución de los hilos 25
Planificación de los hilos 26
Influencia del sistema operativo en la planificación 26
Prioridades de los hilos 26
Métodos para controlar la ejecución de los hilos 27
El método yield() 28
Resumen de la gestión de hilos en Java 28
4 Intercambio de información entre hilos 30
4.1 Intercambio de datos entre hilos 30
4.2 Variables compartidas 32
Soluciones para manejar variables compartidas 33
4.3 Canales de comunicación 33
Clases en Java para la comunicación entre hilos 34
4.3.1 Bloqueos y colas bloqueantes 34
5 Sincronización de hilos 37
5.1 Problemas comunes en la sincronización de hilos 37
Condición de carrera 37
Inconsistencia de memoria 38
Inanición 39
Interbloqueo (deadlock) 39
Bloqueo activo (livelock) 40
5.2 Mecanismos de sincronización de hilos 40
5.2.1 Condiciones de Bernstein 41
5.2.2 Operación atómica 41
5.2.3 Sección crítica 43
5.2.4 Semáforos 43
Métodos de la clase Semaphore 43
5.2.5 Monitores 45
Formas de utilizar monitores en Java 45
5.2.6 Condiciones de espera y notificación 47
5.3 Mecanismos de sincronización avanzados 48

5.3.1 Clase Semaphore 48
5.3.2 Clase Exchanger 48
5.3.3 Clase CountDownLatch 49
5.3.4 Clase CyclicBarrier 50
5.4 Buenas prácticas y estrategias para la sincronización de hilos 51
5.4.1 Minimizar el uso de sincronización 51
Estrategias para minimizar la sincronización 52
5.4.2 Estrategias para evitar el interbloqueo 52
5.4.3 Estrategias para evitar la inanición 52
6 Prioridades de los hilos 53
6.1 Concepto de prioridad en los hilos 53
6.2 Niveles de prioridad 53
6.3 Establecimiento y modificación de prioridades 53
6.4 Efectos de las prioridades durante la ejecución 54
6.5 Buenas prácticas en el uso de prioridades 55
Conclusión 56
7 Bibliografía 57

1 Introducción a la programación multihilo

Resultados de aprendizaje y criterios de evaluación

RESULTADO DE APRENDIZAJE: RA2	
RA2.- Desarrolla aplicaciones compuestas por varios hilos de ejecución analizando y aplicando librerías específicas del lenguaje de programación.	
CRITERIOS DE EVALUACIÓN	
a)	Se han identificado situaciones en las que resulte útil la utilización de varios hilos en un programa.
b)	Se han reconocido los mecanismos para crear, iniciar y finalizar hilos.
c)	Se han programado aplicaciones que implementen varios hilos.
d)	Se han identificado los posibles estados de ejecución de un hilo y programado aplicaciones que los gestionen.
e)	Se han utilizado mecanismos para compartir información entre varios hilos de un mismo proceso.
f)	Se han desarrollado programas formados por varios hilos sincronizados mediante técnicas específicas.
g)	Se ha establecido y controlado la prioridad de cada uno de los hilos de ejecución.
h)	Se han depurado y documentado los programas desarrollados.
i)	Se ha analizado el contexto de ejecución de los hilos
j)	Se han analizado librerías específicas del lenguaje de programación que permiten la programación multihilo.
k)	Se han reconocido los problemas derivados de la compartición de información entre los hilos de un mismo proceso

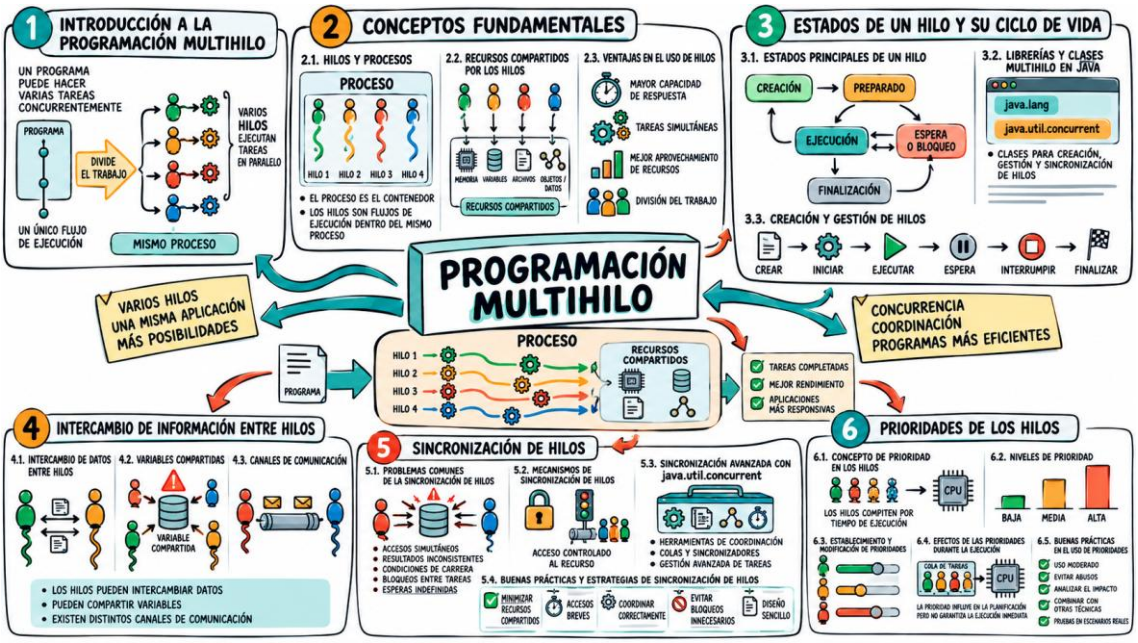
Objetivos de la unidad de trabajo

En la unidad anterior vimos que un programa en ejecución es un **proceso** y que una aplicación puede lanzar varios procesos para repartir el trabajo. En esta unidad damos un paso más: vamos a dividir el trabajo **dentro de un mismo proceso**, en varios flujos de ejecución que llamamos **hilos** (*threads*). Un programa con un solo hilo ejecuta sus instrucciones una detrás de otra y, si una de ellas tarda (leer un fichero grande, esperar una respuesta de la red), todo el programa se queda esperando. Un programa multihilo, en cambio, puede seguir avanzando en otras tareas mientras tanto.

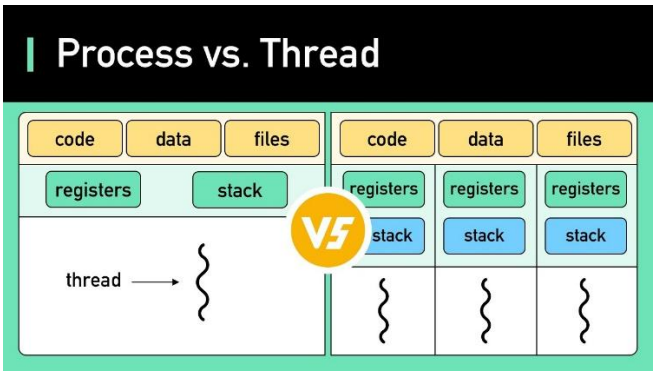
A lo largo de la unidad trabajaremos cuatro grandes objetivos:

- **Conceptos fundamentales**: conocer y comprender los principios básicos de los hilos de ejecución, en qué se diferencian de los procesos, qué recursos comparten y cómo funcionan.
- **Creación y gestión de hilos**: aprender los mecanismos que ofrece Java para crear varios hilos en una aplicación, ponerlos en marcha, esperar a que terminen, detenerlos y conocer los estados por los que pasan.
- **Sincronización y comunicación**: estudiar las técnicas que permiten a los hilos intercambiar información y coordinarse entre sí sin estropear los datos que comparten.
- **Beneficios y desafíos**: identificar las ventajas que aporta la programación multihilo y también los problemas que provoca, para saber cuándo merece la pena utilizarla y cómo evitar sus errores típicos.

El siguiente mapa resume los bloques de la unidad y cómo se relacionan entre sí:



Antes de empezar, puedes ver el siguiente vídeo, que compara de forma visual los procesos y los hilos:



[Enlace a video introductorio]

Ejemplos de la programación multihilo

Aunque no seamos conscientes, casi todas las aplicaciones que usamos a diario son multihilo. Veamos tres ejemplos:



- **Navegador web**: mientras se descarga una imagen o un archivo, podemos seguir desplazándonos por la página, abrir otra pestaña o iniciar una nueva descarga sin esperar a que termine la anterior. Si el navegador tuviera un único hilo, la ventana dejaría de responder al ratón y al teclado durante cada descarga. Con varios hilos, uno se encarga de atender al usuario mientras otros descargan los recursos en segundo plano.



- **Editor de texto**: en un procesador de textos como Microsoft Word, o en el propio IntelliJ IDEA, mientras escribimos, un hilo muestra en pantalla cada tecla que pulsamos, otro revisa la ortografía y la gramática en tiempo real y subraya los errores, y otro guarda automáticamente una copia del documento. Gracias a este reparto, la escritura nunca se interrumpe por las tareas que se hacen de fondo.



- **Videojuegos**: un videojuego tiene que hacer muchas cosas en cada fracción de segundo. Un hilo puede encargarse de dibujar los gráficos (renderizado), otro de la inteligencia artificial de los personajes no jugables (NPC) y otro de la detección de colisiones. Así el juego se ve fluido y responde rápidamente a las acciones del jugador.

En los tres casos se repite la misma idea: hay una tarea que el usuario percibe directamente (la ventana, la escritura, el juego) y otras tareas que pueden hacerse en paralelo sin que el usuario tenga que esperarlas. Repartir esas tareas en hilos distintos es lo que permite que la aplicación siga respondiendo.

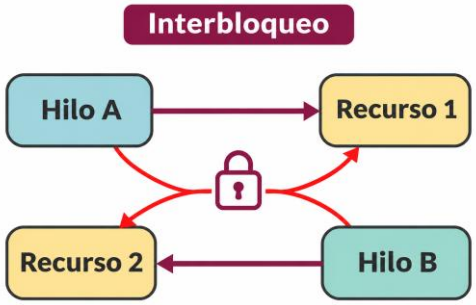
Desafíos de la programación multihilo

Repartir el trabajo entre varios hilos tiene muchas ventajas, pero también plantea retos importantes. La causa de casi todos ellos es la misma: los hilos de un proceso **comparten la memoria**, de modo que varios hilos pueden leer y modificar los mismos datos al mismo tiempo. Los principales desafíos son los siguientes:

- **Condición de carrera**: se produce cuando varios hilos acceden a la vez a datos compartidos y al menos uno de ellos los modifica. El resultado depende del orden en que se intercalen las instrucciones de los hilos, y ese orden cambia en cada ejecución. Por eso el programa puede funcionar bien cien veces y fallar a la ciento una, lo que hace que estos errores sean muy difíciles de reproducir y de corregir.
- **Sección crítica e interbloqueo**: las partes del código en las que se accede a datos compartidos se llaman **secciones críticas**, y su ejecución debe controlarse cuidadosamente para evitar conflictos. Si ese control se diseña mal, puede aparecer un **interbloqueo**: dos o más hilos se quedan esperando indefinidamente un recurso que tiene otro de ellos. Por ejemplo, el hilo A tiene

el recurso 1 y espera el recurso 2, mientras que el hilo B tiene el recurso 2 y espera el recurso 1. Ninguno puede continuar y el programa se queda paralizado.

- **Mecanismos de sincronización**: para evitar estos problemas, Java ofrece herramientas como los **objetos de bloqueo** y los **métodos sincronizados**, que regulan el acceso simultáneo a los recursos compartidos para que los hilos no se estorben entre sí.



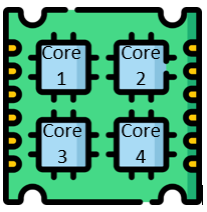
Estos problemas se estudian en detalle en el apartado 5, junto con las técnicas para evitarlos. Por ahora basta con quedarse con una idea: **cuando varios hilos comparten datos, el programador es responsable de coordinar su acceso**.

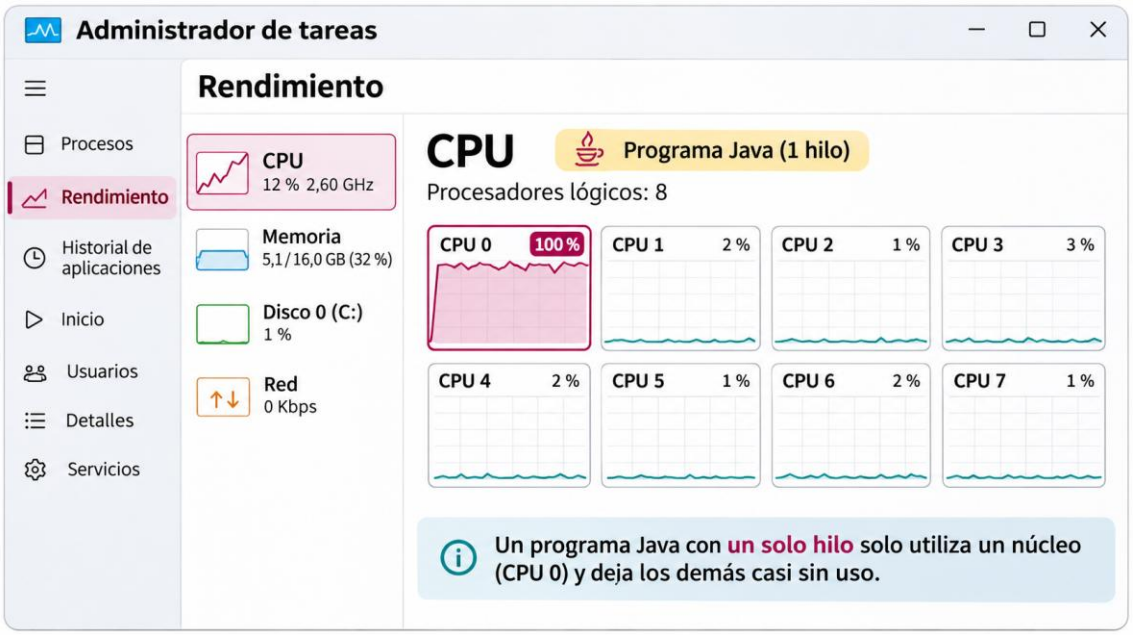
Contexto actual de la programación multihilo

Para entender por qué hoy es imprescindible saber programar con varios hilos, hay que fijarse en cómo han evolucionado los procesadores:

- **Tendencia en los procesadores**: durante muchos años, los ordenadores se hacían más rápidos aumentando la velocidad (frecuencia) de su único núcleo. Hoy los fabricantes han cambiado de enfoque: en lugar de hacer cada núcleo más rápido, **añaden más núcleos** al mismo procesador. Un ordenador de aula actual tiene fácilmente 4, 6 u 8 núcleos.
- **Consumo de energía**: el motivo de este cambio es físico. Subir la frecuencia de un núcleo dispara el consumo eléctrico y el calor que genera. En cambio, varios núcleos funcionando a una frecuencia más moderada ofrecen más rendimiento total consumiendo menos energía.
- **Paralelismo y concurrencia**: con más núcleos, el sistema operativo puede ejecutar más hilos a la vez, uno en cada núcleo, lo que aumenta las posibilidades de realizar tareas concurrentes y de reducir los tiempos de espera.

De aquí se deduce algo muy importante para nosotros como programadores: **un programa con un solo hilo solo puede usar un núcleo, aunque el equipo tenga ocho**. El resto de núcleos quedan sin aprovechar. Para sacar partido al hardware actual no basta con comprar un procesador con más núcleos: hay que programar las aplicaciones con varios hilos.




Administrador de tareas
Rendimiento
Procesos
Rendimiento
Historial de aplicaciones
Inicio
Usuarios
Detalles
Servicios
CPU
12 % 2,60 GHz
Memoria
5,1/16,0 GB (32 %)
Disco 0 (C:)
1 %
Red
0 Kbps
CPU
Programa Java (1 hilo)
Procesadores lógicos: 8
CPU 0 100 %
CPU 1 2 %
CPU 2 1 %
CPU 3 3 %
CPU 4 2 %
CPU 5 1 %
CPU 6 2 %
CPU 7 1 %
Un programa Java con un solo hilo solo utiliza un núcleo (CPU 0) y deja los demás casi sin uso.

2 Conceptos fundamentales

Antes de escribir código multihilo es imprescindible tener claros algunos conceptos: qué es exactamente un hilo, en qué se diferencia de un proceso, qué recursos tiene para él solo y cuáles comparte con los demás hilos. Todo lo que veremos después (creación, sincronización, comunicación) se apoya en estas ideas.

2.1 Hilos y procesos

Un **hilo** (*thread*) es la **unidad básica de ejecución que el sistema operativo planifica en la CPU**. Dicho de otra forma: cuando el sistema operativo decide qué se ejecuta en cada núcleo del procesador, lo que reparte son hilos.

Un hilo **no está ligado a un núcleo concreto**. El sistema operativo puede ejecutarlo en cualquiera de los núcleos disponibles e incluso moverlo de un núcleo a otro mientras se ejecuta, según le convenga para repartir la carga de trabajo.

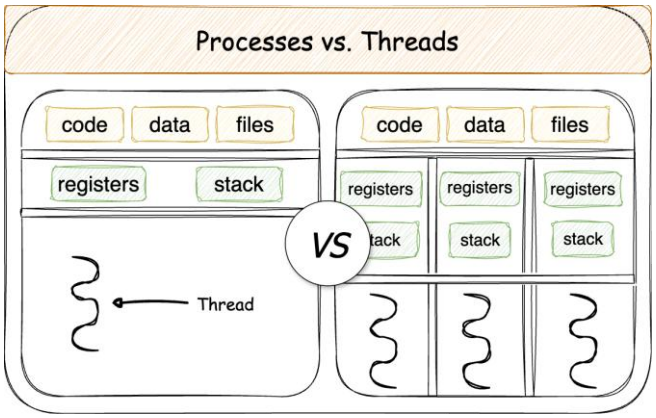
Todo hilo **pertenece a un proceso** y se ejecuta dentro de su contexto. Un proceso tiene como mínimo un hilo: en Java es el **hilo main**, el que ejecuta el método main() al arrancar el programa. A partir de él, el proceso puede crear tantos hilos adicionales como necesite. El sistema operativo gestiona tanto los procesos como sus hilos.

Una forma sencilla de verlo es imaginar un restaurante. El **proceso** es la cocina: tiene sus fogones, sus ingredientes y sus utensilios. Los **hilos** son los cocineros que trabajan en esa cocina: cada uno sigue su propia receta y va por su propio paso, pero todos usan los mismos fogones y los mismos ingredientes. Con un solo cocinero (un solo hilo) los platos salen de uno en uno; con varios, salen varios a la vez, pero tienen que ponerse de acuerdo para no coger la misma sartén al mismo tiempo.

Hilos frente a procesos

Para distinguir bien ambos conceptos conviene fijarse en tres aspectos:

- **Procesos independientes**: los procesos son entidades independientes y cada uno cuenta con su propio espacio de memoria. Como vimos en la unidad anterior, un proceso no puede leer ni modificar directamente las variables de otro.
- **Hilos que comparten recursos**: dentro de un mismo proceso, los hilos comparten la memoria y los recursos de ese proceso.
- **Ejecución concurrente**: los hilos permiten que un proceso avance en varias tareas a la vez, ya sea **alternándose en un mismo núcleo** (lo que llamamos **concurrencia**) o ejecutándose **simultáneamente en núcleos distintos** (lo que llamamos **paralelismo**).



2.2 Recursos compartidos por los hilos

Como los hilos se ejecutan dentro de un mismo proceso, combinan dos tipos de elementos: unos que son **propios de cada hilo**, y que le permiten llevar su propio camino de ejecución sin interferir con los demás, y otros que son **comunes a todo el proceso**.

Recursos que NO comparten los hilos

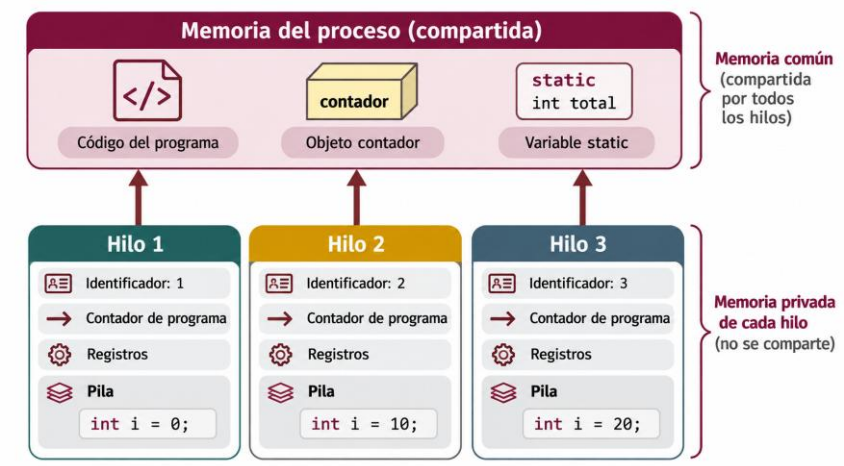
Cada hilo dispone, solo para él, de:

- **Un identificador único**, que lo distingue de los demás hilos del proceso.
- **Su propio contador de programa**, que indica por qué instrucción va su ejecución. Gracias a él, dos hilos pueden estar ejecutando el mismo método pero en líneas distintas.
- **Su propio conjunto de registros** de la CPU. Cuando el sistema operativo retira a un hilo del procesador para dar paso a otro, guarda sus registros; cuando le vuelve a tocar, los restaura y el hilo continúa exactamente donde lo dejó.
- **Su propia pila de ejecución**, donde se guardan sus **variables locales** y la secuencia de **llamadas a métodos**. Por eso, una variable local declarada dentro de un método es privada del hilo que la ejecuta: los demás hilos no la ven, aunque ejecuten el mismo método.

Recursos que SÍ comparten los hilos

Todos los hilos de un mismo proceso comparten:

- **El código**: todos acceden a las mismas clases y métodos del programa.
- **Los datos**: comparten los objetos que hay en la memoria del proceso, es decir, los **atributos** de los objetos y las **variables static**. Si dos hilos tienen una referencia al mismo objeto, los dos pueden leer y modificar sus atributos. Así pueden intercambiar información directamente, sin necesidad de los mecanismos de comunicación entre procesos (flujos de entrada y salida, ficheros…) que usábamos en la unidad anterior.
- **Los recursos del sistema** que el sistema operativo ha concedido al proceso, como los **archivos abiertos** o las **conexiones de red**.



Riesgos de compartir

Compartir la memoria es muy cómodo y rápido, pero también supone un riesgo: **un hilo puede modificar datos compartidos mientras otro hilo los está usando**, y eso afecta al resultado de los demás. Para evitarlo hay que tener en cuenta tres ideas que desarrollaremos a lo largo de la unidad:

1. **Bloqueo de recursos**: los hilos deben utilizar mecanismos de bloqueo para controlar el acceso a los recursos compartidos, de forma que no se estorben al modificarlos.
2. **Comunicación entre hilos**: cuando un hilo necesita un dato que produce otro, ambos deben coordinarse para no leer un dato incompleto o antiguo.
3. **Técnicas de gestión**: Java ofrece herramientas específicas para gestionar de forma eficiente los recursos compartidos. Las estudiaremos en los apartados 4 y 5.

2.3 Ventajas en el uso de hilos

Utilizar varios hilos dentro de un proceso, en lugar de lanzar varios procesos independientes, aporta las siguientes ventajas:

VENTAJA	DESCRIPCIÓN
Capacidad de respuesta	Los hilos permiten que un proceso siga respondiendo al usuario aunque esté realizando una tarea larga. Es lo que vimos con el navegador: un hilo atiende la ventana mientras otros descargan. También es la base de los servidores: un hilo recibe las peticiones y cada petición se atiende en otro hilo, de modo que se pueden atender muchos clientes a la vez.
Compartición de recursos	Los hilos de un mismo proceso comparten la memoria y los recursos, lo que facilita mucho la comunicación entre ellos: basta con que accedan al mismo objeto. A cambio, como pueden modificar los datos a la vez, hay que sincronizarlos.
Mejor uso de la CPU	Mientras un hilo espera a algo lento (leer del disco, recibir datos por la red o que el usuario pulse una tecla), otros hilos del mismo proceso pueden seguir usando la CPU. El procesador no se queda parado esperando.
Eficiencia en el uso de memoria	Como los hilos comparten la memoria del proceso, crear un hilo es mucho más económico, en memoria y en tiempo, que crear un proceso nuevo. Además, el cambio de contexto entre hilos de un mismo proceso es más rápido que entre procesos, porque el sistema operativo no tiene que cambiar de espacio de memoria.

VENTAJA	DESCRIPCIÓN
Paralelismo real	En un procesador multinúcleo, varios hilos del mismo proceso se ejecutan a la vez en núcleos distintos. La mejora real depende de dos cosas: de que la tarea se pueda dividir en partes independientes y de cuánto tengan que sincronizarse los hilos entre sí. Repartir un trabajo entre cuatro hilos no lo hace necesariamente cuatro veces más rápido.

¿Cuándo utilizar hilos en una aplicación?

Resulta especialmente aconsejable utilizar varios hilos en estas situaciones:

- **Manejo de entradas y salidas de varios dispositivos**: cuando la aplicación debe atender varias fuentes a la vez (el usuario, la red, el disco) sin quedarse bloqueada esperando a una de ellas.
- **Realización de tareas en paralelo**: cuando la aplicación debe realizar diferentes tareas a la vez para ser más eficiente.
- **Priorización de tareas**: cuando hay tareas críticas, como atender al usuario, que no deben quedar retrasadas por otras tareas de fondo, como generar un informe o hacer una copia de seguridad.
- **Entorno multiprocesador**: cuando la aplicación se ejecuta en un equipo con varios núcleos (hoy, prácticamente cualquiera) y la tarea se puede repartir entre ellos.

3 Estados de un hilo y su ciclo de vida

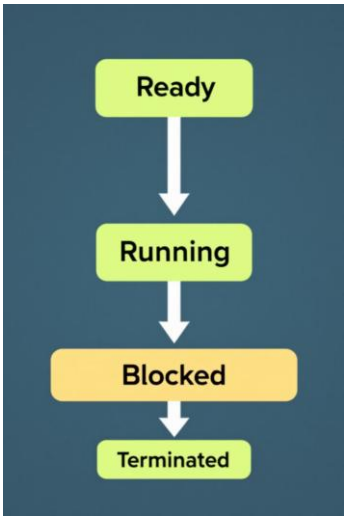
Desde que se crea hasta que termina, un hilo pasa por varios **estados**. El conjunto de estados y de cambios entre ellos se llama **ciclo de vida** del hilo. El hilo cambia de estado según las condiciones del sistema (por ejemplo, si hay un núcleo libre) y según la lógica del propio programa (por ejemplo, si el código le manda esperar).

Conocer estos estados es importante por dos motivos: nos ayuda a entender qué está haciendo cada hilo en un momento dado y nos permite averiguar, cuando un programa se queda 'colgado', qué hilo está esperando y a qué.

3.1 Estados principales de un hilo

De forma general, un hilo puede encontrarse en cuatro estados:

- **Nuevo**: el hilo se ha creado (existe el objeto Thread), pero todavía no ha empezado a ejecutarse porque no se ha llamado a su método start().
- **Ejecutable o en ejecución**: ya se ha llamado a start(). El hilo puede estar ejecutándose en un núcleo o estar listo, esperando a que el sistema operativo le asigne uno. Java no distingue entre estas dos situaciones: para él, ambas son el mismo estado.
- **No ejecutable o bloqueado**: el hilo no puede ejecutarse temporalmente porque está esperando algo. Puede estar esperando un **bloqueo** que tiene otro hilo (al intentar entrar en un bloque synchronized ocupado), un **aviso** de otro hilo (wait(), join()) o a que **pase un tiempo** (sleep()).
- **Terminado o muerto**: el hilo ha finalizado su ejecución y ya no puede volver a ejecutarse.



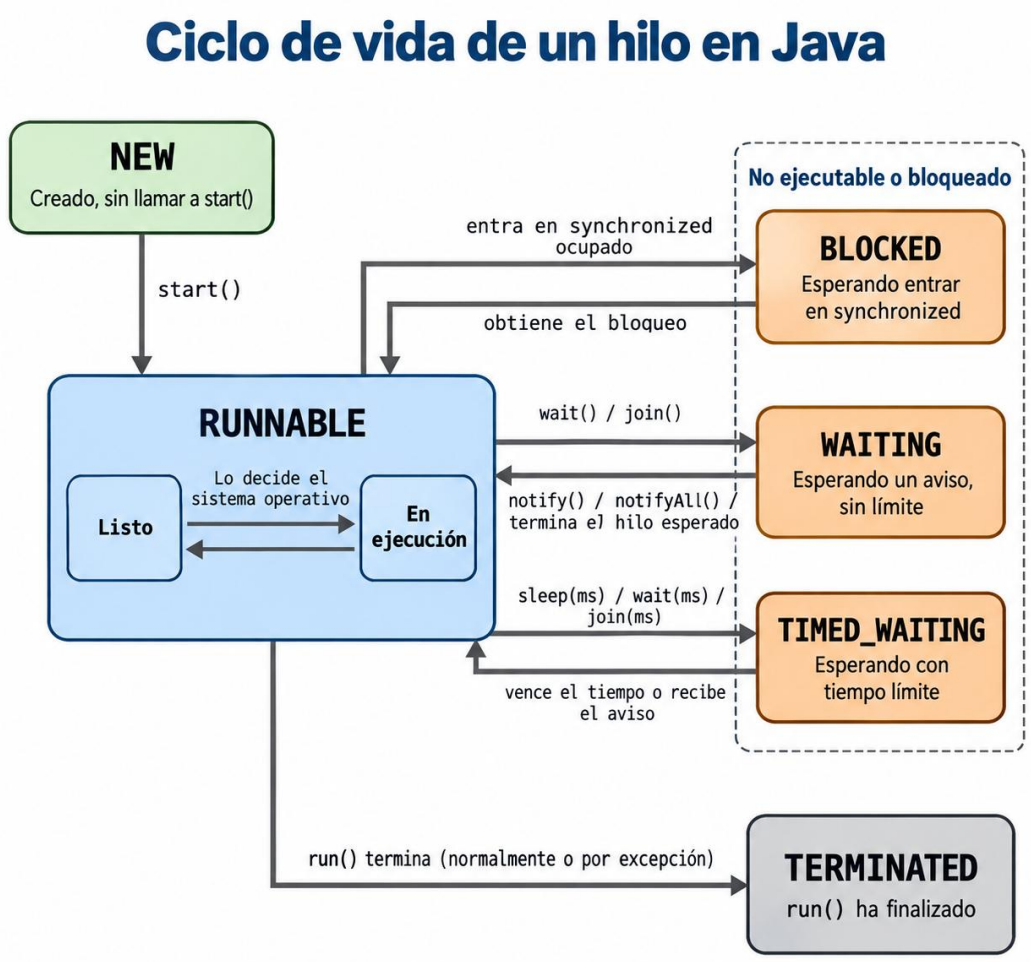
Transición entre estados

Veamos ahora qué provoca que un hilo pase de un estado a otro:

- **Estado nuevo**: el hilo se ha creado con new, pero no ha comenzado a ejecutarse. Permanece en este estado hasta que se llama a su método **start()**, que lo pasa a ejecutable.

- **Estado ejecutable o en ejecución**: el hilo está listo para ejecutarse y puede estar en ejecución activa o esperando un momento a que haya un procesador libre. El planificador del sistema operativo lo va poniendo y quitando del procesador sin que el hilo cambie de estado para Java.
- **No ejecutable o bloqueado**: el hilo pasa a este estado cuando se encuentra con una espera: un bloqueo que tiene otro hilo, un aviso que todavía no ha llegado (wait(), join()) o un tiempo que todavía no ha pasado (sleep()). **Vuelve a ser ejecutable** cuando se libera el bloqueo, recibe el aviso o vence el tiempo.
- **Terminado o muerto**: el hilo termina cuando su método run() finaliza, ya sea de forma normal (llega a la última instrucción) o por una excepción que nadie ha capturado. Al terminar, el hilo libera su pila, pero la memoria y los archivos que compartía siguen perteneciendo al proceso y los demás hilos los pueden seguir usando. Un hilo terminado **no puede volver a iniciarse**: si se llama otra vez a start() sobre él, Java lanza una IllegalThreadStateException. Si se necesita repetir la tarea, hay que crear un hilo nuevo.

El siguiente diagrama resume el ciclo de vida completo de un hilo en Java, con los estados que veremos a continuación y las llamadas que provocan cada cambio:



Método getState()

El método **getState()** de la clase Thread permite consultar el estado de un hilo en cualquier momento. Mientras que en teoría agrupamos todas las esperas en un único estado 'bloqueado', Java distingue el motivo de la espera y devuelve uno de estos **seis valores**:

VALOR	SIGNIFICADO
NEW	Creado, sin llamar todavía a start().
RUNNABLE	En ejecución o listo para ejecutarse. Java también deja en RUNNABLE a un hilo que está esperando una operación de entrada/salida (por ejemplo, leer de teclado).
BLOCKED	Esperando para entrar en un bloque o método synchronized que tiene ocupado otro hilo.
WAITING	Esperando sin tiempo límite: ha llamado a wait() o a join() sin parámetros.
TIMED_WAITING	Esperando con tiempo límite: está en sleep(), o en wait() o join() con un tiempo máximo.
TERMINATED	El método run() ha finalizado.

El siguiente programa crea un hilo que se duerme medio segundo y va consultando su estado desde el hilo main:

```
public class EstadosHilo {


    public static void main(String[] args) throws InterruptedException {
        Thread hilo = new Thread(() -> {
            try {
                Thread.sleep(500);   // el hilo se duerme medio segundo
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });


        System.out.println("Antes de start(): " + hilo.getState());   // NEW
        hilo.start();
        System.out.println("Justo tras start(): " + hilo.getState()); // RUNNABLE
        Thread.sleep(100);
        System.out.println("Mientras duerme: " + hilo.getState());   // TIMED_WAITING
        hilo.join();                                                 // main espera a que termine
        System.out.println("Tras join(): " + hilo.getState());       // TERMINATED
    }
}
```

Una posible salida es:

```
Antes de start(): NEW
Justo tras start(): RUNNABLE
Mientras duerme: TIMED_WAITING
Tras join(): TERMINATED
```

Explicación del ejemplo

- Antes de llamar a start(), el hilo solo es un objeto: su estado es **NEW**.
- Justo después de start(), lo normal es ver **RUNNABLE**. Si el hilo ha sido muy rápido y ya ha llegado a su sleep(), podríamos ver TIMED_WAITING: el resultado depende del planificador.
- Pasados 100 milisegundos, el hilo está dentro de su sleep(500): el estado es **TIMED_WAITING**.

- Con join(), el hilo main espera a que el otro termine. Después, el estado es **TERMINATED**.

Diferencias entre hilos y procesos

Aunque los hilos y los procesos tienen ciclos de vida parecidos, hay diferencias importantes entre ellos:

- Los hilos **comparten la memoria y los recursos** del proceso, mientras que los procesos tienen **recursos propios**.
- Precisamente por compartir, los hilos requieren una **gestión más compleja de la sincronización** y del bloqueo de los recursos compartidos.
- Los procesos son **independientes**, mientras que los hilos **dependen del contexto del proceso** al que pertenecen: si el proceso termina, terminan todos sus hilos.

Diferencias entre los estados de los hilos y de los procesos

En la unidad anterior estudiamos el diagrama de estados de un proceso. Si lo comparamos con el de los hilos, vemos que los procesos tienen estados relacionados con la **gestión de la memoria** del sistema operativo, como *suspendido listo* y *suspendido bloqueado*, que aparecen cuando el proceso entero se lleva a memoria secundaria (*swap*). Los hilos no tienen esos estados. En cambio, el estado 'bloqueado' de los hilos se divide en tres (BLOCKED, WAITING y TIMED_WAITING) según el **motivo de la espera**, porque esas esperas están ligadas a la sincronización entre hilos que comparten datos.



Diagrama de estados de un proceso (unidad de trabajo 1)

3.2 Librerías y clases multihilo en Java

Java ofrece una amplia variedad de herramientas para desarrollar aplicaciones concurrentes. Se organizan principalmente en dos paquetes:

- **java.lang**: contiene las clases básicas para crear hilos y sincronizarlos. No hace falta importarlo, porque Java lo importa automáticamente en cualquier programa.
- **java.util.concurrent**: añadido en Java 5, ofrece herramientas de más alto nivel que simplifican mucho la programación multihilo y evitan errores habituales. Incluye los subpaquetes java.util.concurrent.atomic y java.util.concurrent.locks.

3.2.1 Utilidades de concurrencia en java.lang

El paquete java.lang incluye varias clases e interfaces fundamentales para trabajar con hilos:

1. **Clase Thread**: representa un hilo de ejecución. Proporciona los métodos esenciales para crear y controlar su ciclo de vida, como start() (iniciarlo), sleep() (dormirlo un tiempo) y join() (esperar a que termine).
2. **Interfaz Runnable**: representa una tarea que puede ejecutar un hilo. Permite añadir la funcionalidad de un hilo a una clase sin necesidad de extender la clase Thread. Como tiene un único método, run(), puede escribirse de forma muy compacta como una **expresión lambda**: () -> { … }.
3. **Clase ThreadGroup**: agrupa varios hilos para tratarlos como una unidad (por ejemplo, para interrumpirlos todos a la vez), lo que facilita su control conjunto. Es una clase **heredada** de las primeras versiones de Java: hoy, para gestionar varios hilos, se usa ExecutorService, que veremos en el apartado 3.2.2.
4. **Clase Object**: la clase de la que heredan todos los objetos de Java proporciona tres métodos fundamentales para coordinar hilos: wait(), notify() y notifyAll(). Solo pueden llamarse **dentro de un bloque synchronized sobre ese mismo objeto**. Si no, Java lanza una IllegalMonitorStateException.

Ejemplo de uso del paquete java.lang

El siguiente programa crea tres hilos dentro de un mismo grupo. Los tres esperan hasta que el hilo main les avisa de que la tarea está disponible. Todavía no hemos estudiado wait() y notifyAll() (lo haremos en el apartado 4.1); por ahora basta con entender la idea general.

```
public class EjemploThreadGroupBreve {

    private static final Object monitor = new Object();
    private static boolean tareaDisponible = false;

    public static void main(String[] args) {
        // Crear un grupo de hilos
        ThreadGroup grupo = new ThreadGroup("GrupoEjemplo");

        // Crear e iniciar hilos dentro del grupo
        for (int i = 1; i <= 3; i++) {
            new Thread(grupo, new Tarea(), "Hilo " + i).start();
        }

        // Simular preparación de la tarea
        try {
            Thread.sleep(500);
            synchronized (monitor) {
                tareaDisponible = true;
                monitor.notifyAll(); // Notifica a todos los hilos
```

```
                System.out.println("Hilo principal: Tarea disponible, "
                        + "notificación enviada.");
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    // Clase que representa la tarea que espera hasta ser notificada
    static class Tarea implements Runnable {
        @Override
        public void run() {
            synchronized (monitor) {
                while (!tareaDisponible) {
                    try {
                        System.out.println(Thread.currentThread().getName()
                                + " espera la tarea...");
                        monitor.wait(); // Espera hasta recibir la notificación
                    } catch (InterruptedException e) {
                        Thread.currentThread().interrupt();
                    }
                }
                System.out.println(Thread.currentThread().getName()
                        + " ha comenzado su tarea.");
            }
        }
    }
}
```

Explicación del ejemplo

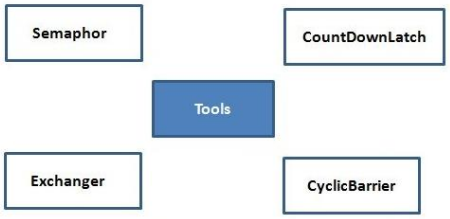
- **ThreadGroup**: se crea un grupo llamado 'GrupoEjemplo' y se crean tres hilos (Hilo 1, Hilo 2 y Hilo 3) dentro de él, pasando el grupo como primer argumento del constructor de Thread. El segundo argumento es la tarea (un Runnable) y el tercero, el nombre del hilo.
- **Sincronización con wait() y notifyAll()**: cada hilo entra en el bloque synchronized (monitor), ve que tareaDisponible es false y se queda esperando en monitor.wait(). Medio segundo después, el hilo main pone tareaDisponible a true y llama a monitor.notifyAll(), que despierta a los tres hilos para que continúen.

3.2.2 Utilidades de concurrencia en java.util.concurrent

El paquete java.util.concurrent está diseñado para facilitar el desarrollo de aplicaciones multihilo complejas. Ofrece herramientas ya hechas y optimizadas que resuelven los problemas más habituales, de modo que no tengamos que programarlos a mano. Sus componentes se agrupan así:

Clases de sincronización

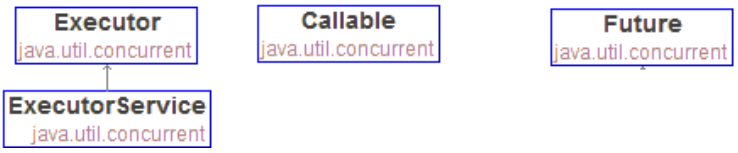
CLASE	PARA QUÉ SIRVE
Semaphore (semáforo)	Controla el acceso a un recurso compartido mediante un conjunto de permisos: solo pueden usar el recurso a la vez tantos hilos como permisos haya.
CountDownLatch (contador de espera)	Permite que un hilo espere a que se complete un conjunto de operaciones en otros hilos.
CyclicBarrier (barrera cíclica)	Hace que un conjunto de hilos se esperen unos a otros en un punto determinado antes de seguir.
Exchanger (intercambiador)	Permite que dos hilos intercambien datos en un punto de encuentro.



Las estudiaremos en detalle en el apartado 5.3.

Interfaces para ejecutar tareas

- **Executor y ExecutorService**: ejecutan tareas mediante un **grupo reutilizable de hilos**, lo que se conoce como *pool* de hilos. En lugar de crear un hilo nuevo para cada tarea, enviamos las tareas al pool y sus hilos las van ejecutando. Los pools se crean con la clase **Executors**; por ejemplo, Executors.newFixedThreadPool(3) crea un pool con tres hilos.
- **Callable y Future**: **Callable** es una tarea, parecida a Runnable, pero que **devuelve un resultado** y puede lanzar excepciones. Al enviarla a un ExecutorService, este nos devuelve un objeto **Future**, que es como un resguardo: con su método **get()** recogemos el resultado cuando la tarea termina.
- **Separación de la lógica**: estas interfaces separan **qué hace** la tarea (Runnable o Callable) de **cómo y en qué hilo se ejecuta** (ExecutorService).



El siguiente ejemplo envía a un pool una tarea que calcula la suma de los números del 1 al 100 y recoge el resultado:

```
import java.util.concurrent.*;

public class SumaConFuture {

    public static void main(String[] args) throws InterruptedException, ExecutionException {
        ExecutorService pool = Executors.newFixedThreadPool(2);

        // Tarea que devuelve un resultado: la suma de 1 a 100
        Callable<Integer> sumar = () -> {
            int suma = 0;
            for (int i = 1; i <= 100; i++) {
                suma += i;
            }
            return suma;
        };

        Future<Integer> resultado = pool.submit(sumar);  // la ejecuta un hilo del pool
        System.out.println("El hilo main sigue con lo suyo mientras se calcula la suma...");

        int suma = resultado.get();   // si la tarea no ha terminado, main espera
        System.out.println("Resultado de la tarea: " + suma);

        pool.shutdown();   // sin shutdown() los hilos del pool siguen vivos
    }
}
```

Explicación del ejemplo

- La tarea se define como un **Callable<Integer>**: igual que un Runnable, pero su método devuelve un Integer.
- pool.submit(sumar) envía la tarea al pool y devuelve **inmediatamente** un Future<Integer>. El hilo main no espera: sigue ejecutando la siguiente línea.
- resultado.get() devuelve el resultado. Si la tarea todavía no ha terminado, el hilo main se queda esperando en esa línea hasta que termine.
- pool.shutdown() cierra el pool cuando ha terminado sus tareas. Si se olvida, el programa no termina, porque los hilos del pool siguen vivos esperando nuevas tareas.

Gestión de colas concurrentes (BlockingQueue)

- **Estructuras de datos especializadas**: BlockingQueue es una interfaz de **cola segura para varios hilos**. Su método **put()** espera si la cola está llena y su método **take()** espera si está vacía. Sus implementaciones más usadas son las clases **ArrayBlockingQueue** (de tamaño fijo) y **LinkedBlockingQueue**.
- **Procesamiento de tareas**: son la base del modelo **productor-consumidor**: unos hilos dejan datos o tareas en la cola y otros los recogen. Lo veremos en el apartado 4.3.
- **Otras implementaciones**: SynchronousQueue, PriorityBlockingQueue y DelayQueue, cada una con características específicas.

Paquete java.util.concurrent.atomic

Contiene **variables atómicas**, como **AtomicInteger** y **AtomicLong**, que aseguran la integridad de los datos en operaciones concurrentes. Sus operaciones, como **incrementAndGet()**, se realizan de forma **indivisible**: ningún otro hilo puede intervenir a mitad de la operación. Gracias a ello, un contador compartido por varios hilos es seguro sin necesidad de usar synchronized. Lo veremos en el apartado 5.2.2.

Paquete java.util.concurrent.locks

Ofrece alternativas a la palabra clave synchronized que proporcionan mayor control sobre la sincronización:

- **Interfaz Lock**: permite una gestión más fina de los bloqueos. El bloqueo se toma y se libera de forma **explícita**, con **lock()** y **unlock()**. Además, **tryLock()** permite intentar tomarlo sin quedarse esperando indefinidamente. Su implementación más habitual es la clase ReentrantLock.
- **Interfaz ReadWriteLock**: pensada para recursos que se leen mucho y se modifican poco. Permite que **varios hilos lean a la vez**, pero **solo uno escriba**, y únicamente cuando nadie está leyendo.

Con Lock, el unlock() se coloca siempre en un bloque finally, para que el bloqueo se libere aunque se produzca una excepción:

```
Lock cerrojo = new ReentrantLock();

cerrojo.lock();            // coge el bloqueo (espera si lo tiene otro hilo)
```

```
try {
    contador++;             // sección crítica
} finally {
    cerrojo.unlock();       // se libera siempre, aunque haya una excepción
}
```

Ejemplo integrador con java.util.concurrent

El siguiente programa combina varias de estas herramientas: un pool de tres hilos (ExecutorService), un semáforo de dos permisos, un CountDownLatch y una CyclicBarrier. No te preocupes si ahora no entiendes todos los detalles; volverás a este ejemplo cuando hayas estudiado el apartado 5.

```
import java.util.concurrent.*;

public class ConcurrenciaEjemplo {

    public static void main(String[] args) {
        // Creación de un semáforo que permite el acceso simultáneo de hasta 2 hilos
        Semaphore semaphore = new Semaphore(2);

        // Creación de un CountDownLatch con un conteo de 3 (esperar a 3 tareas)
        CountDownLatch latch = new CountDownLatch(3);

        // Creación de una barrera cíclica para sincronizar 3 hilos en un punto común
        CyclicBarrier barrier = new CyclicBarrier(3, () ->
            System.out.println("Todos los hilos han llegado a la barrera, continuando..."));

        // Uso de ExecutorService para gestionar tareas con un pool de hilos
        ExecutorService executor = Executors.newFixedThreadPool(3);

        // Ejemplo de tarea usando Semaphore, CountDownLatch y CyclicBarrier
        Runnable tarea = () -> {
            try {
                // Adquiere un permiso del semáforo para acceder al recurso
                semaphore.acquire();
                System.out.println(Thread.currentThread().getName()
                        + " ha adquirido el semáforo.");

                // Simulación de trabajo accediendo al recurso
                Thread.sleep(1000);

                // Decrementa el contador del CountDownLatch
                latch.countDown();
                System.out.println(Thread.currentThread().getName()
                        + " ha completado la tarea y disminuye el latch.");

                // Libera el permiso ANTES de la barrera: así puede entrar el tercer hilo
                semaphore.release();
                System.out.println(Thread.currentThread().getName()
                        + " ha liberado el semáforo.");

                // Llega al punto de sincronización de la barrera cíclica
                System.out.println(Thread.currentThread().getName()
                        + " está esperando en la barrera.");
                barrier.await();

            } catch (InterruptedException | BrokenBarrierException e) {
                Thread.currentThread().interrupt();
                System.out.println(Thread.currentThread().getName()
                        + " ha sido interrumpido.");
            }
        };

        // Enviar tareas al ExecutorService
        for (int i = 0; i < 3; i++) {
            executor.submit(tarea);
```

```
        }

        // Espera a que todas las tareas completen su trabajo usando el latch
        try {
            latch.await(); // Espera hasta que el latch llegue a cero
            System.out.println("Todas las tareas han completado su parte. "
                    + "Continuando en el hilo principal.");
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            System.out.println("El hilo principal ha sido interrumpido.");
        }

        // Apaga el ExecutorService
        executor.shutdown();
    }
}
```

Explicación del ejemplo

- El **pool** ejecuta tres veces la misma tarea, cada vez en uno de sus tres hilos.
- El **semáforo** solo tiene dos permisos, así que al principio solo dos hilos 'trabajan' y el tercero espera en acquire().
- Cada hilo, al terminar su trabajo, descuenta uno al **CountDownLatch**. Cuando llega a cero, el hilo main, que esperaba en latch.await(), continúa.
- Después, cada hilo libera su permiso y espera en la **barrera** a los demás. Cuando llegan los tres, se ejecuta la acción de la barrera y todos continúan.
- **Ojo con el orden**: el permiso se libera **antes** de esperar en la barrera. Si se liberara después, los dos primeros hilos se quedarían esperando en la barrera con su permiso cogido, el tercero nunca conseguiría un permiso para llegar a ella y el programa se quedaría parado para siempre. Es un ejemplo de **interbloqueo**, uno de los problemas que estudiaremos en el apartado 5.1.

3.3 Creación y gestión de hilos

3.3.1 Creación de hilos en Java

Todo programa Java comienza su ejecución en un único hilo, el **hilo main**, que la máquina virtual de Java (JVM) crea automáticamente para ejecutar el método main(). Desde ese hilo podemos crear otros hilos para realizar tareas en paralelo. Existen dos formas principales de hacerlo:

- **Extendiendo la clase Thread**.
- **Implementando la interfaz Runnable**.

En ambos casos, el código que debe ejecutar el nuevo hilo se escribe en el método **run()**, y el hilo se pone en marcha llamando a **start()**.

Creación de hilos extendiendo la clase Thread

Es la forma más directa. Los pasos son:

1. **Crear una nueva clase** que extienda la clase Thread.
2. **Sobrescribir el método run()**, escribiendo dentro el código que debe ejecutar el hilo.

3. **Instanciar y ejecutar**: crear un objeto de la nueva clase e iniciar el hilo invocando su método start().

```
class Saludo extends Thread {
    @Override
    public void run() {
        System.out.println("¡Hola desde el hilo!");
    }


    public static void main(String[] args) {
        Saludo hilo1 = new Saludo();
        hilo1.start();
    }
}
```

Al ejecutarlo, el hilo hilo1 imprime su mensaje de saludo.

Siempre start(), nunca run(). El método start() es el que crea el hilo nuevo, y es ese hilo nuevo quien llama a run(). Si llamamos nosotros directamente a hilo1.run(), no se crea ningún hilo: el código de run() se ejecuta en el hilo main, como cualquier otro método. El programa parecerá funcionar, pero no habrá ninguna ejecución en paralelo.

Creación de hilos implementando la interfaz Runnable

La interfaz Runnable permite dar a una clase la funcionalidad de un hilo sin necesidad de heredar de Thread. Esto es útil cuando la clase ya extiende de otra, porque en Java una clase solo puede heredar de una única clase. Por eso este método es más flexible y general. Los pasos son:

1. **Declarar la clase** indicando que implementa la interfaz Runnable.
2. **Implementar el método run()** con el código que debe ejecutar el hilo.
3. **Crear una instancia** de la clase que implementa Runnable.
4. **Crear un objeto Thread**, pasando la instancia de Runnable como argumento del constructor.
5. **Iniciar el hilo** llamando a start() sobre el objeto Thread.

```
class SaludoRunnable implements Runnable {
    @Override
    public void run() {
        System.out.println("¡Hola desde el hilo Runnable!");
    }


    public static void main(String[] args) {
        SaludoRunnable saludo = new SaludoRunnable();
        Thread hilo1 = new Thread(saludo);
        hilo1.start();
    }
}
```

La clase SaludoRunnable implementa Runnable y redefine el método run() para imprimir un mensaje. Fíjate en el reparto de papeles: el objeto saludo representa **la tarea** (qué hay que hacer), mientras que hilo1 es **el hilo** que la ejecuta. Al llamar a hilo1.start(), el nuevo hilo ejecuta el run() de saludo.

Como Runnable solo tiene un método, cuando la tarea es corta se suele escribir directamente como una expresión lambda, sin crear una clase aparte:

```java
public class SaludoLambda {
    public static void main(String[] args) {
        Thread hilo1 = new Thread(() -> System.out.println("¡Hola desde una lambda!"));
        hilo1.start();
    }
}
```

Elección entre Thread y Runnable

- **Opción Runnable**: permite que la clase tenga la funcionalidad de un hilo sin limitar su herencia, porque la clase queda libre para extender de otra.
- **Extender Thread**: es la forma más directa cuando la clase no hereda de ninguna otra. Sobrescribir otros métodos de Thread, aparte de run(), casi nunca es necesario.
- **Flexibilidad**: implementar Runnable es, en general, la opción recomendada por su mayor flexibilidad.

¿Cuándo usar cada método?

- **Usar Runnable**: si tu clase ya extiende otra clase o buscas mayor flexibilidad. Al separar la tarea del hilo, puedes pasar la misma tarea a varios hilos o a un pool (ExecutorService).
- **Usar Thread**: si tu clase no necesita heredar de ninguna otra y buscas simplicidad. Es adecuado para aplicaciones pequeñas o pruebas rápidas.

En resumen, **implementa Runnable siempre que sea posible**, ya que es la opción más flexible y modular.

3.3.2 Gestión y planificación de hilos

Una vez creados varios hilos, necesitamos entender cómo se reparten el procesador y qué métodos ofrece Java para controlar su ejecución: dormirlos un tiempo, esperar a que otro hilo termine, interrumpirlos o comprobar si siguen vivos.



Tipos de ejecución de los hilos

- **Paralelismo**: en procesadores con varios núcleos, cada núcleo puede ejecutar un hilo distinto **al mismo tiempo**.
- **Pseudoparalelismo**: cuando hay más hilos que núcleos, el sistema operativo los va **alternando** tan rápido que parecen simultáneos. Esto es la **concurrencia**.

En la práctica se dan las dos cosas a la vez: un equipo con 8 núcleos ejecuta 8 hilos en paralelo, pero normalmente tiene cientos de hilos de todos los programas abiertos, que se van turnando en esos 8 núcleos.



Planificación de los hilos

En Java actual, **cada hilo de Java es un hilo del sistema operativo**. Por tanto, es el **planificador del sistema operativo** (*scheduler*) quien decide qué hilo se ejecuta, cuándo y en qué núcleo. Su funcionamiento se basa en tres ideas:

1. **La prioridad es solo una sugerencia**: el planificador puede tener en cuenta la prioridad del hilo, pero Java no garantiza que el de mayor prioridad se ejecute antes.
2. **Turnos de CPU**: los hilos listos se reparten el procesador por turnos de tiempo, de forma parecida a un *round-robin*: cada uno trabaja un rato y deja paso al siguiente.
3. **Planificación apropiativa**: el sistema operativo puede quitarle el procesador a un hilo en cualquier momento (porque se acaba su turno o para atender a otro), sin que el hilo lo pida. Por eso **el orden de ejecución de los hilos no es predecible**: si ejecutas dos veces el mismo programa multihilo, los mensajes pueden salir en un orden distinto cada vez. Un programa correcto no debe depender de ese orden.

Influencia del sistema operativo en la planificación

- **Time-slicing**: en sistemas como Windows o Linux, cada hilo se ejecuta durante un intervalo corto de tiempo, llamado **quantum**, para evitar que un hilo monopolice el procesador. Al terminar su quantum, el sistema operativo da paso a otro hilo.
- **Sistemas sin time-slicing**: hoy todos los sistemas operativos de escritorio y de móvil usan time-slicing. Los sistemas en los que un hilo se ejecuta hasta terminar o hasta ceder voluntariamente el procesador (planificación **cooperativa**) son históricos o muy específicos, como algunos sistemas embebidos.

Prioridades de los hilos



Cada hilo de Java tiene una **prioridad** entre **1** (mínima) y **10** (máxima). El hilo main tiene prioridad **5** (Thread.NORM_PRIORITY), y cada hilo nuevo **hereda** la prioridad del hilo que lo crea.

- El método **getPriority()** devuelve la prioridad actual de un hilo.

- El método **setPriority(int priority)** establece la prioridad de un hilo entre 1 y 10. Si el valor está fuera de ese rango, se lanza una IllegalArgumentException. Lo habitual es usar las constantes Thread.MIN_PRIORITY (1), Thread.NORM_PRIORITY (5) y Thread.MAX_PRIORITY (10).

El programador puede cambiar la prioridad de un hilo, pero es **solo una sugerencia** al sistema operativo: un programa correcto nunca debe depender de ella. Lo estudiaremos en detalle en el apartado 6.

Métodos para controlar la ejecución de los hilos



La clase Thread ofrece varios métodos para controlar la ejecución de los hilos. Los podemos agrupar en tres categorías:

1. Suspensión y espera

- **join()**: hace que el hilo que lo llama **espere hasta que otro hilo termine**. Por ejemplo, si el hilo main ejecuta hilo1.join(), el main se detiene en esa línea hasta que hilo1 termine. Es útil cuando un hilo necesita los resultados de otro.
- **Thread.sleep(long millis)**: **duerme** al hilo que lo ejecuta durante los milisegundos indicados. Mientras duerme, su estado es TIMED_WAITING y no consume CPU. Obliga a capturar la excepción InterruptedException, porque otro hilo puede despertarlo antes de tiempo.

2. Interrupción

- **interrupt()**: **pide** a un hilo que se detenga activando su **marca de interrupción**, pero no lo para a la fuerza. Si el hilo está en sleep(), join() o wait(), se despierta con una InterruptedException.
- **isInterrupted()**: permite al hilo comprobar su marca de interrupción. Lo habitual es usarla en el bucle de trabajo para terminar de forma ordenada: while (!Thread.currentThread().isInterrupted()) { … }.
- **Responsabilidad del programador**: es el programador quien decide cómo responde un hilo a una interrupción; normalmente, terminando su ejecución. Hay un detalle importante: **al capturar InterruptedException, la marca de interrupción se borra**. Por eso, en el catch se vuelve a activar con Thread.currentThread().interrupt(); si no, el bucle no se enteraría de que le han pedido parar.

3. Verificación de estado

- **isAlive()**: devuelve true si el hilo se ha iniciado y todavía no ha terminado, aunque en ese momento esté dormido o bloqueado. Es útil para **consultar sin esperar** si un hilo sigue trabajando (por ejemplo, para mostrar 'Descargando…'). Para **esperar** a que termine se usa join().

El siguiente ejemplo reúne estos métodos. El hilo main deja trabajar a un hilo durante un segundo, le pide que pare y espera a que termine:

```
public class JoinInterrupt {
```

```
    public static void main(String[] args) throws InterruptedException {
        Thread trabajador = new Thread(() -> {
            int vuelta = 0;
            while (!Thread.currentThread().isInterrupted()) {
                vuelta++;
                System.out.println("Trabajador: vuelta " + vuelta);
                try {
                    Thread.sleep(300);
                } catch (InterruptedException e) {
                    // la excepción borra la marca: la volvemos a activar
                    Thread.currentThread().interrupt();
                }
            }
            System.out.println("Trabajador: me han pedido parar, termino ordenadamente.");
        });


        trabajador.start();
        Thread.sleep(1000);                  // dejamos que trabaje un segundo
        System.out.println("main: isAlive() = " + trabajador.isAlive());
        trabajador.interrupt();              // le pedimos que pare
        trabajador.join();                   // esperamos a que termine de verdad
        System.out.println("main: isAlive() = " + trabajador.isAlive());
    }
}
```

Una posible salida es:

```
Trabajador: vuelta 1
Trabajador: vuelta 2
Trabajador: vuelta 3
Trabajador: vuelta 4
main: isAlive() = true
Trabajador: me han pedido parar, termino ordenadamente.
main: isAlive() = false
```

Explicación del ejemplo

- El trabajador repite su bucle **mientras no le interrumpan**, durmiendo 300 milisegundos en cada vuelta.
- El hilo main duerme un segundo, comprueba con **isAlive()** que el trabajador sigue vivo y le llama a **interrupt()**.
- Lo más probable es que el trabajador esté dentro de su sleep(), así que se despierta con una InterruptedException. En el catch vuelve a activar la marca; al volver a la condición del while, ve que está interrumpido y sale del bucle.
- El main espera con **join()** a que el trabajador termine de verdad. Después, isAlive() ya devuelve false.

El método yield()

- **Ceder el turno**: Thread.yield() indica al planificador que el hilo actual puede ceder su turno a otros hilos. El hilo no se bloquea: sigue en estado RUNNABLE.
- **Funcionamiento no garantizado**: es solo una pista; el planificador puede ignorarla y devolverle el control inmediatamente. En la práctica casi no se usa: para esperar se usan sleep(), join() o los mecanismos de sincronización.

Resumen de la gestión de hilos en Java

- **Manipulación de prioridades**: las prioridades son solo una sugerencia al sistema operativo; un programa correcto no debe depender de ellas.

- **Influencia del sistema operativo**: en Java actual es el sistema operativo quien planifica los hilos; por eso el orden de ejecución no es predecible.
- **Métodos de administración**: sleep(), join(), interrupt() e isAlive() permiten controlar la ejecución de los hilos de forma segura.

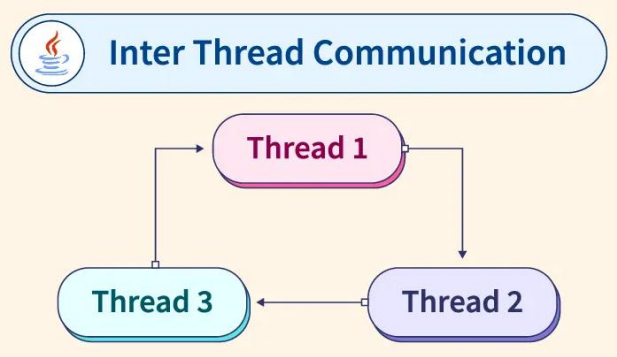
4 Intercambio de información entre hilos

El intercambio de datos entre hilos es fundamental en la programación multihilo: permite que los hilos **colaboren** para resolver una tarea de manera sincronizada. Este proceso asegura la coordinación entre ellos y evita conflictos al acceder a los recursos compartidos.

Para intercambiar información correctamente hay que garantizar dos cosas: que el hilo que necesita un dato **sepa cuándo está listo** (para no leerlo antes de tiempo) y que **nadie lo modifique a la vez** mientras se usa.

Java ofrece tres formas de hacerlo, de la más manual a la más cómoda:

1. **wait() / notify()**: un hilo espera y otro le avisa cuando el dato está listo (apartado 4.1).
2. **Variables compartidas** protegidas con volatile, con clases atómicas o con synchronized (apartado 4.2).
3. **Colas bloqueantes**, que hacen la espera y el aviso por nosotros (apartado 4.3).



4.1 Intercambio de datos entre hilos

Java proporciona varios métodos fundamentales para la comunicación entre hilos. No están en la clase Thread, sino en la clase **java.lang.Object**, así que cualquier objeto puede usarse como punto de encuentro entre hilos:

1. **wait()**: el hilo se detiene (estado WAITING) y **suelta el bloqueo** del objeto, para que otro hilo pueda entrar y cambiar el dato que está esperando.
2. **wait(long tiempo)**: igual que wait(), pero si pasan esos milisegundos sin recibir ningún aviso, el hilo se despierta solo (estado TIMED_WAITING).
3. **notify()**: despierta a **uno** de los hilos que están esperando en ese objeto.
4. **notifyAll()**: despierta a **todos** los hilos que están esperando en ese objeto.



Para usarlos correctamente hay que respetar dos reglas:

- Los tres métodos **solo funcionan dentro de un bloque o método synchronized sobre el mismo objeto**. Fuera de él lanzan una IllegalMonitorStateException.
- El wait() va siempre **dentro de un while (condición)**, nunca dentro de un if. Al despertar, el hilo debe **volver a comprobar la condición**, porque puede que otro hilo se le haya adelantado y la condición vuelva a no cumplirse.

Ejemplo básico de comunicación entre hilos

El siguiente ejemplo muestra cómo utilizar wait() y notify() para intercambiar datos entre un hilo productor y un hilo consumidor. El almacén **solo guarda un dato**: el productor espera a que el consumidor lo recoja, y el consumidor espera a que haya uno.

```
class Almacen {
    private int dato;
    private boolean disponible = false;

    public synchronized void producir(int valor) {
        while (disponible) {
            try {
                wait(); // Espera a que el consumidor consuma el dato
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }
        }
        dato = valor;
        disponible = true;
        System.out.println("Producido: " + valor);
        notify(); // Notifica al consumidor que el dato está disponible
    }

    public synchronized int consumir() {
        while (!disponible) {
            try {
                wait(); // Espera a que el productor produzca un dato
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return -1;
            }
        }
        disponible = false;
        System.out.println("Consumido: " + dato);
        notify(); // Notifica al productor que puede producir otro dato
        return dato;
    }
}

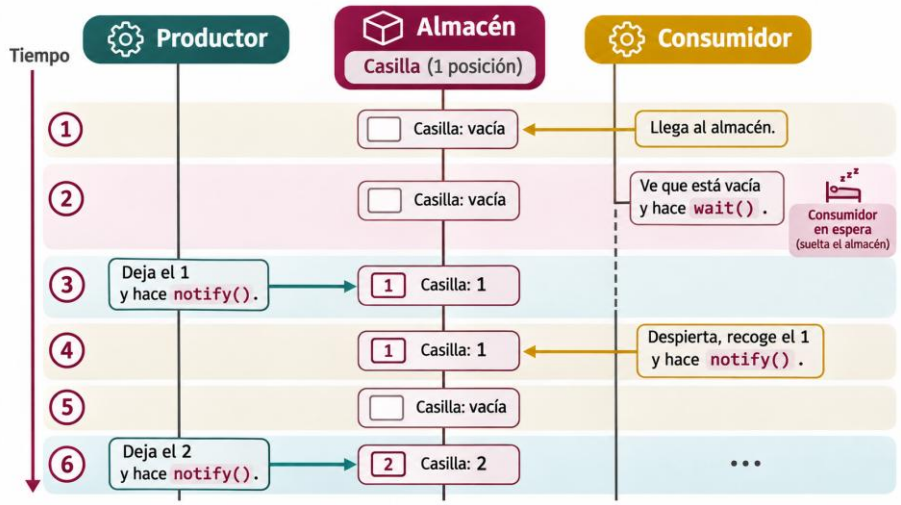
public class ComunicacionEntreHilos {
    public static void main(String[] args) {
        Almacen almacen = new Almacen();
        // Hilo productor
        Thread productor = new Thread(() -> {
            for (int i = 1; i <= 5; i++) {
                almacen.producir(i);
            }
        });
        // Hilo consumidor
        Thread consumidor = new Thread(() -> {
            for (int i = 1; i <= 5; i++) {
                almacen.consumir();
            }
        });
        productor.start();
        consumidor.start();
    }
}
```

La salida es siempre la misma, porque los dos hilos se van turnando gracias a wait() y notify():

```
Producido: 1
Consumido: 1
Producido: 2
Consumido: 2
...
Producido: 5
Consumido: 5
```

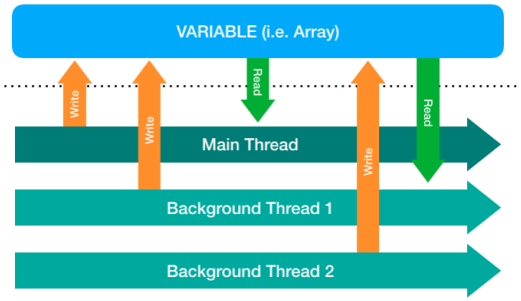
Explicación del ejemplo

- La clase **Almacen** tiene un dato y un indicador disponible que dice si hay un dato pendiente de recoger. Sus dos métodos son synchronized, así que solo un hilo puede estar dentro del almacén a la vez.
- **producir()**: mientras haya un dato sin recoger, el productor espera con wait(). Cuando el consumidor lo recoge y le avisa, guarda el nuevo dato, pone disponible a true y avisa con notify().
- **consumir()**: mientras no haya dato, el consumidor espera con wait(). Cuando lo hay, lo recoge, pone disponible a false y avisa al productor con notify().
- Si un hilo es interrumpido mientras espera, vuelve a activar su marca de interrupción y sale del método.



4.2 Variables compartidas

Las **variables compartidas** son variables accesibles por varios hilos del mismo proceso (atributos de un objeto que comparten o variables static). Se usan para **comunicar información** entre hilos y para **coordinar tareas**.



Su uso plantea dos problemas, que estudiaremos a fondo en el apartado 5.1:

- **Condiciones de carrera**: si dos hilos hacen likes++ a la vez, se puede perder uno de los incrementos.
- **Problemas de visibilidad**: un hilo cambia una variable y otro sigue viendo el valor antiguo.

Por eso es esencial garantizar que las operaciones con variables compartidas sean seguras y coherentes.

Soluciones para manejar variables compartidas

MECANISMO	QUÉ GARANTIZA	CUÁNDO USARLO
volatile	Que las actualizaciones de la variable sean visibles inmediatamente para todos los hilos.	Cuando un hilo escribe la variable y los demás solo la leen (por ejemplo, un indicador de parada).
Clases atómicas (AtomicInteger…)	Operaciones seguras e indivisibles sobre una variable, sin necesidad de synchronized.	Contadores y operaciones sobre una sola variable (incrementar, sumar…).
Bloques synchronized	Que solo un hilo a la vez ejecute el bloque, y que los cambios hechos dentro sean visibles para los demás.	Cuando hay que actualizar varias variables juntas o hacer más de un paso seguido.

Veamos un ejemplo de **volatile**. El hilo main pone parar a true pasado un segundo, y el otro hilo debe verlo y terminar:

```
public class VolatileParar {


    // Prueba a quitar 'volatile': el hilo puede no ver el cambio y no terminar
    private static volatile boolean parar = false;


    public static void main(String[] args) throws InterruptedException {
        Thread hilo = new Thread(() -> {
            long vueltas = 0;
            while (!parar) {
                vueltas++;
            }
            System.out.println("Hilo: he visto parar = true tras " + vueltas + " vueltas");
        });


        hilo.start();
        Thread.sleep(1000);
        parar = true;                          // main cambia la variable compartida
        System.out.println("main: he puesto parar = true");
        hilo.join();
    }
}
```

Explicación del ejemplo

- El hilo da vueltas en un bucle mientras parar sea false. Pasado un segundo, el hilo main pone parar = true.
- Gracias a **volatile**, el cambio se ve inmediatamente y el hilo termina.
- **Prueba a quitar volatile**: lo más probable es que el programa no termine nunca, porque el hilo sigue viendo el valor antiguo de parar. Tendrás que detenerlo con el botón rojo **Stop** de IntelliJ. Este es el problema de visibilidad que veremos como inconsistencia de memoria en el apartado 5.1.

4.3 Canales de comunicación

Usar wait() y notify() a mano funciona, pero es fácil equivocarse. Los **canales de comunicación** ofrecen una alternativa más cómoda:

1. **Intercambio estructurado**: permiten que los hilos intercambien datos de manera organizada y eficiente.
2. **Evitar problemas comunes**: proporcionan mecanismos seguros que evitan las condiciones de carrera y las inconsistencias de memoria.
3. **Herramientas esenciales**: son fundamentales para la colaboración y el trabajo en paralelo entre hilos.

Clases en Java para la comunicación entre hilos

El paquete java.util.concurrent proporciona clases diseñadas específicamente para facilitar la comunicación entre hilos:

- **BlockingQueue** (cola bloqueante): para el modelo productor-consumidor. Es la que vamos a usar.
- **Exchanger**: dos hilos se intercambian un objeto a la vez.
- **CyclicBarrier**: varios hilos esperan a estar todos en un punto antes de seguir.

Exchanger y CyclicBarrier se estudian en el apartado 5.3.

4.3.1 Bloqueos y colas bloqueantes

Las **colas bloqueantes** permiten que un hilo añada elementos mientras otro los consume, y **gestionan automáticamente la sincronización** entre ellos. Sus dos operaciones clave son:

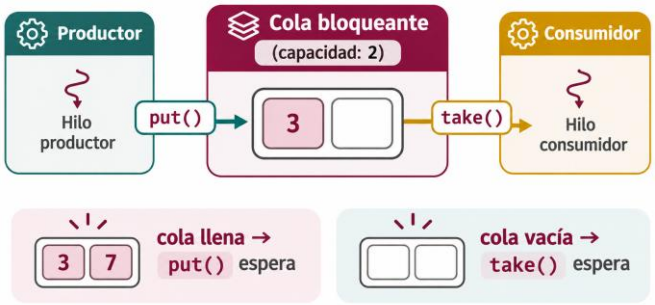
- **put(x)**: añade un elemento. Si la cola está llena, el hilo espera a que haya hueco.
- **take()**: saca un elemento. Si la cola está vacía, el hilo espera a que llegue uno.

Por dentro, hacen el wait() y el notify() que en el apartado 4.1 escribíamos a mano. Por eso son mucho más cómodas y seguras.

Las implementaciones más comunes son:

- **LinkedBlockingQueue**: el tamaño máximo es opcional.
- **ArrayBlockingQueue**: tiene un tamaño fijo que se indica al crearla.
- **PriorityBlockingQueue**: saca primero el elemento de mayor prioridad, y no tiene límite de tamaño.

Las colas bloqueantes son ideales para el **problema del productor-consumidor**, en el que unos hilos producen datos y otros los consumen a su propio ritmo.



Ejemplo: Productor-Consumidor

En el siguiente ejemplo, el productor deja cinco números en una cola en la que **solo caben dos**, y el consumidor, que es más lento, los va recogiendo uno por segundo:

```
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;


public class ProductorConsumidorCola {


    public static void main(String[] args) {
        BlockingQueue<Integer> cola = new ArrayBlockingQueue<>(2);    // caben 2


        Thread productor = new Thread(() -> {
            try {
                for (int i = 1; i <= 5; i++) {
                    cola.put(i);                // espera si la cola está llena
                    System.out.println("Productor: dejo " + i
                            + "   (en cola: " + cola.size() + ")");
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });


        Thread consumidor = new Thread(() -> {
            try {
                for (int i = 1; i <= 5; i++) {
                    Thread.sleep(1000);         // el consumidor es más lento
                    int valor = cola.take();    // espera si la cola está vacía
                    System.out.println("    Consumidor: recojo " + valor);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });


        productor.start();
        consumidor.start();
    }
}
```

Una posible salida es:

```
Productor: dejo 1   (en cola: 1)
Productor: dejo 2   (en cola: 2)
Productor: dejo 3   (en cola: 2)
    Consumidor: recojo 1
    Consumidor: recojo 2
Productor: dejo 4   (en cola: 2)
    Consumidor: recojo 3
Productor: dejo 5   (en cola: 2)
    Consumidor: recojo 4
    Consumidor: recojo 5
```

Explicación del ejemplo

- El productor deja el 1 y el 2 enseguida. Al intentar dejar el 3, la cola está llena y **put() le hace esperar**.
- Cada vez que el consumidor recoge un número con take(), queda un hueco y el productor puede dejar el siguiente. Por eso la cola nunca tiene más de dos elementos.
- Si el consumidor fuera más rápido que el productor, sería él quien esperaría en take() a que hubiera algún número.

- Fíjate en que no hemos escrito ningún synchronized, wait() ni notify(): la cola se encarga de todo. Los mensajes pueden salir en un orden ligeramente distinto en cada ejecución.

5 Sincronización de hilos

Como hemos visto, los hilos se comunican intercambiando información a través de variables y objetos en memoria. Como todos los hilos pertenecen al mismo proceso, pueden acceder a toda su memoria, y esa es la forma más eficiente de comunicarse.

Sin embargo, cuando varios hilos manipulan a la vez objetos compartidos, pueden aparecer **resultados erróneos** o el programa puede **quedarse paralizado**. La solución a estos problemas es la **sincronización**.

5.1 Problemas comunes en la sincronización de hilos

Estos son los problemas más habituales que aparecen al trabajar con hilos que comparten datos:

PROBLEMA	EN POCAS PALABRAS
Condición de carrera	El resultado depende del orden en que varios hilos acceden a la memoria compartida.
Inconsistencia de memoria	Un hilo cambia una variable y otro sigue viendo el valor antiguo.
Inanición	Un hilo nunca consigue el recurso, aunque este se libere, porque siempre se lo lleva otro hilo.
Interbloqueo	Dos o más hilos se bloquean mutuamente esperando recursos que tiene el otro.
Bloqueo activo	Como el interbloqueo, ningún hilo avanza; pero aquí no están parados: no dejan de reaccionar el uno al otro y gastan CPU sin progresar.

Condición de carrera



Una **condición de carrera** ocurre cuando el resultado de un programa depende del **orden en que los hilos acceden a la memoria compartida**. Sucede cuando varios hilos acceden a la vez a la misma variable y al menos uno de ellos la modifica.

El ejemplo típico es un contador. Supongamos que dos hilos hacen likes++ sobre una variable que vale 100. Esperaríamos 102, pero podemos obtener 101. La razón es que likes++ no es una única operación, sino **tres pasos**: leer el valor, sumarle 1 y escribir el resultado. Si un hilo se cuela en medio de los pasos del otro, uno de los incrementos se pierde:

Tiempo	Hilo 1	Hilo 2	likes en memoria
t1	Lee 100 (`likes = 100`).	–	100
t2	–	Lee 100 (`likes = 100`).	100
t3	Suma y escribe 101 (`likes = 101`).	–	101
t4	–	Suma y escribe 101 (`likes = 101`).	101

Se esperaba 102: **se ha perdido un like.**

El siguiente programa lo demuestra. Dos hilos suman 100 000 likes cada uno sobre la misma variable:

```
public class CarreraLikes {

    private static int likes = 0;    // variable compartida por los dos hilos

    public static void main(String[] args) throws InterruptedException {
        Runnable darLikes = () -> {
            for (int i = 0; i < 100_000; i++) {
                likes++;             // leer, sumar 1 y escribir: tres pasos
            }
        };

        Thread hilo1 = new Thread(darLikes);
        Thread hilo2 = new Thread(darLikes);
        hilo1.start();
        hilo2.start();
        hilo1.join();
        hilo2.join();

        System.out.println("Likes esperados: 200000");
        System.out.println("Likes obtenidos: " + likes);
    }
}
```

Una posible salida es:

```
Likes esperados: 200000
Likes obtenidos: 133795
```

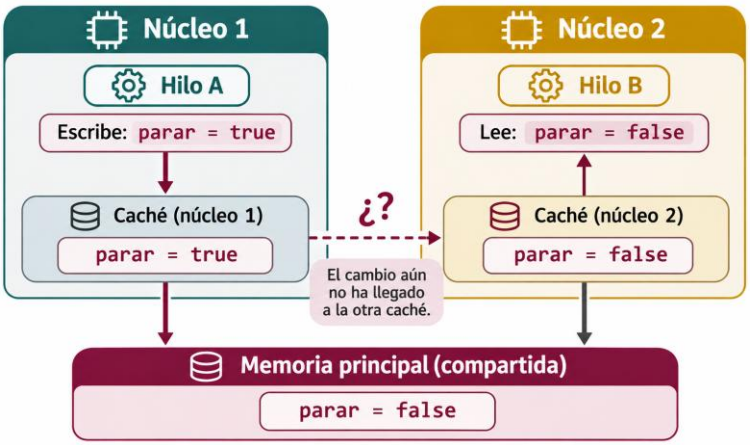
El resultado es incorrecto y, además, **cambia en cada ejecución**. Este tipo de problema aparece en las **secciones críticas** del código, que requieren un acceso exclusivo (**exclusión mutua**). En el apartado 5.2 veremos cómo solucionarlo.

Inconsistencia de memoria



La **inconsistencia de memoria** se produce cuando **un hilo cambia una variable y otro hilo sigue viendo el valor antiguo**.

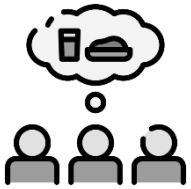
Ocurre porque cada núcleo del procesador trabaja con su propia **memoria caché**, y además el compilador puede **reordenar instrucciones** para optimizar el programa. Por eso, el cambio que hace un hilo no siempre llega a tiempo a la memoria compartida, o el otro hilo sigue leyendo su copia de la caché.



Lo vimos en el ejemplo VolatileParar del apartado 4.2: un hilo pone parar = true y el otro sigue en su while (!parar) para siempre.

Solución: usar **volatile** (si un hilo escribe y los otros solo leen) o **synchronized** (que además de dar exclusión mutua garantiza que se vean los cambios).

Inanición



La **inanición** se produce cuando un hilo se ve **constantemente privado de acceder a un recurso compartido**.

Ocurre porque el reparto no es justo. Por ejemplo, con synchronized, cuando un hilo libera el bloqueo, puede volver a cogerlo el mismo hilo una y otra vez, y otro hilo que esperaba no llega a entrar nunca. Es como la cola de una discoteca en la que siempre pasan otros antes que tú.

Este problema es difícil de detectar, porque el programa no se para: simplemente, un hilo no avanza. Requiere atención especial en el diseño del programa para asegurar que todos los hilos tengan oportunidades justas de acceder a los recursos.

Interbloqueo (deadlock)



El **interbloqueo** o *deadlock* es un problema crítico que aparece cuando dos o más hilos se quedan **esperando indefinidamente** porque cada uno espera un recurso que tiene otro, formando un **ciclo de dependencia** que impide que ninguno avance.

Puede no presentarse en todas las ejecuciones, pero cuando ocurre, los hilos quedan completamente bloqueados y el programa se paraliza. En el siguiente ejemplo, el hilo A coge el recurso 1 y luego quiere el 2, mientras que el hilo B coge el recurso 2 y luego quiere el 1:

```
public class Interbloqueo {

    private static final Object recurso1 = new Object();
    private static final Object recurso2 = new Object();

    public static void main(String[] args) {
        Thread hiloA = new Thread(() -> {
            synchronized (recurso1) {
                System.out.println("Hilo A: tengo el recurso 1 y quiero el 2");
                dormir(100);
                synchronized (recurso2) {
                    System.out.println("Hilo A: tengo los dos recursos");
                }
            }
        });

        Thread hiloB = new Thread(() -> {
            synchronized (recurso2) {
                System.out.println("Hilo B: tengo el recurso 2 y quiero el 1");
                dormir(100);
                synchronized (recurso1) {
                    System.out.println("Hilo B: tengo los dos recursos");
                }
            }
        });

        hiloA.start();
        hiloB.start();
    }
```

```
    private static void dormir(long ms) {
        try {
            Thread.sleep(ms);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

Una posible salida es:

```
Hilo A: tengo el recurso 1 y quiero el 2
Hilo B: tengo el recurso 2 y quiero el 1
```

Y el programa **se queda parado para siempre**: hay que detenerlo con el botón **Stop** de IntelliJ. El sleep(100) solo sirve para asegurar que los dos hilos tengan tiempo de coger su primer recurso; sin él, el interbloqueo se produciría solo algunas veces, lo que lo haría todavía más difícil de detectar.

Cómo detectarlo en IntelliJ: mientras el programa está parado, en la ventana **Run** pulsa el botón **Dump Threads** (icono de cámara). IntelliJ muestra el estado de todos los hilos en ese momento: verás los dos hilos en estado BLOCKED y, en la traza de cada uno, qué objeto tiene bloqueado (locked) y cuál está esperando (waiting to lock). Así se descubre el ciclo de espera.

Cómo evitarlo: que todos los hilos pidan los recursos **en el mismo orden**, retener los recursos **el menor tiempo posible** y **no esperar a otros hilos mientras se tiene un recurso** que ellos necesitan. En el ejemplo, bastaría con que el hilo B pidiera también primero el recurso 1 y después el 2. Lo veremos en el apartado 5.4.2.

Bloqueo activo (livelock)

El **bloqueo activo** es similar al interbloqueo, pero los hilos afectados **no están parados**.



Cada hilo cede para dejar pasar al otro, y el otro hace exactamente lo mismo a la vez. Es como dos personas que se cruzan en un pasillo y se apartan siempre hacia el mismo lado: se mueven todo el rato, pero ninguna consigue pasar.

Igual que en el interbloqueo, ningún hilo progresa. La diferencia es que aquí los hilos siguen ejecutándose (estado RUNNABLE) y **gastan CPU sin hacer nada útil**.

5.2 Mecanismos de sincronización de hilos

Los problemas anteriores aparecen porque los hilos acceden a los datos compartidos de forma desordenada. Para evitarlos, hay que controlar el acceso a esos datos:

- **Ejecución asíncrona**: los hilos pueden trabajar en paralelo, sin coordinarse, cuando **no comparten** recursos críticos.
- **Ejecución síncrona**: los hilos deben **sincronizar** su ejecución cuando acceden a datos compartidos.

- **Controlar el acceso**: los mecanismos de sincronización garantizan la **exclusión mutua**: solo un hilo a la vez dentro de la sección crítica.
- **Evitar problemas**: así se evitan las condiciones de carrera y las inconsistencias de memoria.

En Java disponemos de: **synchronized**, **wait()/notify()**, **Lock**, **Semaphore**, las **clases atómicas** y las demás clases de **java.util.concurrent**.

5.2.1 Condiciones de Bernstein

Las **condiciones de Bernstein** permiten saber si dos fragmentos de código (segmentos) pueden ejecutarse en paralelo sin interferir entre sí. Para cada segmento distinguimos sus **variables de entrada** (las que lee) y sus **variables de salida** (las que escribe). Dos segmentos i y j pueden ejecutarse a la vez si se cumplen estas tres condiciones:

1. **Dependencia de flujo**: las variables de entrada del segmento j deben ser distintas de las variables de salida del segmento i. Así, j no depende de lo que calcule i.
2. **Antidependencia**: las variables de entrada del segmento i deben ser distintas de las variables de salida de j. Así, i no depende de los resultados de j.
3. **Dependencia de salida**: las variables de salida de i deben ser distintas de las de salida de j. Así, el resultado no depende de qué segmento se ejecute el último.

Veamos un ejemplo con tres instrucciones:

SEGMENTO	ENTRADA (LEE)	SALIDA (ESCRIBE)
S1: a = x + y;	x, y	a
S2: b = x * 2;	x	b
S3: x = a + 1;	a	x

- **S1 y S2** pueden ejecutarse en paralelo: los dos leen x, pero ninguno la escribe, y escriben variables distintas.
- **S1 y S3** no pueden: S3 lee a, que es la salida de S1 (dependencia de flujo), y además S3 escribe x, que S1 lee (antidependencia).
- **S2 y S3** tampoco: S3 escribe x, que S2 lee. Según cuál se ejecute antes, b valdrá una cosa u otra.

Los segmentos que cumplen estas condiciones pueden ejecutarse de forma **asíncrona**. Para los que no las cumplen, hay que usar **mecanismos de sincronización**.

5.2.2 Operación atómica

Una **operación atómica** es una operación que **se realiza en su totalidad sin ser interrumpida**: ningún otro hilo puede ver ni modificar los datos a mitad de la operación.

- **Atomicidad en sistemas monoprocesador**: en un sistema con un solo procesador, una operación que se ejecuta en una sola instrucción de CPU se considera atómica. Pero cuidado: **una sola línea de Java puede ser varias instrucciones de CPU**. Por eso x++ no es atómica ni siquiera con un solo núcleo.

- **Desafíos en sistemas multiprocesador**: con varios procesadores, asegurar la atomicidad es más complicado, porque varios hilos pueden acceder a los mismos datos exactamente al mismo tiempo.
- **Atomicidad en Java**: leer o escribir un int, un boolean o una referencia es atómico. Pero **contador++ no lo es**, porque son tres pasos (leer, sumar y escribir), **aunque la variable sea volatile**: volatile garantiza que se vea el último valor, no que los tres pasos se hagan sin interrupción. Para operaciones atómicas se usan **AtomicInteger**, **AtomicLong**… o **synchronized**.

El siguiente ejemplo resuelve el problema del contador de likes del apartado 5.1 usando un AtomicInteger:

```
import java.util.concurrent.atomic.AtomicInteger;

public class OperacionAtomicaEjemplo {

    // AtomicInteger permite operaciones atómicas sobre la variable 'contador'
    private static AtomicInteger contador = new AtomicInteger(0);

    public static void main(String[] args) {
        // Crear dos hilos que incrementarán el contador
        Thread hilo1 = new Thread(new Incrementador());
        Thread hilo2 = new Thread(new Incrementador());

        // Iniciar los hilos
        hilo1.start();
        hilo2.start();

        // Esperar a que ambos hilos terminen
        try {
            hilo1.join();
            hilo2.join();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }

        // Imprimir el valor final del contador
        System.out.println("Valor final del contador: " + contador.get());
    }

    // Clase interna que implementa Runnable para incrementar el contador
    static class Incrementador implements Runnable {
        @Override
        public void run() {
            for (int i = 0; i < 100_000; i++) {
                contador.incrementAndGet(); // Incrementa el contador de manera atómica
            }
        }
    }
}
```

La salida es siempre la correcta:

```
Valor final del contador: 200000
```

Explicación del ejemplo

- **AtomicInteger contador = new AtomicInteger(0)**: declaramos el contador como variable atómica, inicializada a 0.

- **contador.incrementAndGet()**: incrementa el valor en uno y devuelve el nuevo valor, todo como **una única operación atómica**. Ningún otro hilo puede colarse en medio.
- Los dos hilos suman 100 000 cada uno y, tras esperarlos con join(), el contador vale exactamente 200 000. Compáralo con el resultado de CarreraLikes, donde se perdían incrementos.

5.2.3 Sección crítica



Una **sección crítica** es un fragmento de código en el que se accede a variables o recursos compartidos y que, por tanto, debe ejecutarse de forma controlada. En el ejemplo CarreraLikes, la sección crítica es la línea likes++.

Una solución correcta al problema de la sección crítica debe cumplir tres requisitos:

- **Exclusión mutua**: solo un hilo puede estar dentro de la sección crítica a la vez.
- **Progreso**: si nadie está en la sección crítica y algún hilo quiere entrar, alguno tiene que poder entrar. No se puede dejar la puerta cerrada sin motivo.
- **Espera limitada**: un hilo que pide entrar no puede quedarse esperando para siempre. Hay un límite de veces que otros hilos pueden colarse antes que él. Este requisito evita la inanición.

Java proporciona dos herramientas principales para implementar secciones críticas: los **semáforos** y los **monitores**.

5.2.4 Semáforos



Los **semáforos** son una herramienta de sincronización que controla el acceso a recursos con un **número limitado de instancias**, por ejemplo, un aparcamiento con un número fijo de plazas.

Un semáforo es un **contador de permisos** con dos operaciones:

- **Coger un permiso**: si el contador es 0, el hilo espera; si no, le resta 1 y pasa. En Java es el método **acquire()**.
- **Soltar un permiso**: suma 1 al contador y, si había algún hilo esperando, lo deja pasar. En Java es el método **release()**.

No hay que confundir estas operaciones con el wait() de la clase Object: son mecanismos distintos.

En Java, los semáforos se implementan con la clase **Semaphore** del paquete java.util.concurrent. Con un solo permiso, new Semaphore(1), el semáforo funciona como un **cerrojo**: solo pasa un hilo cada vez. A este tipo de semáforo se le llama **semáforo binario** o *mutex* (de exclusión mutua).

Métodos de la clase Semaphore

MÉTODO	DEVUELVE	QUÉ HACE
new Semaphore(int permisos)	(constructor)	Crea el semáforo con ese número de permisos.

MÉTODO	DEVUELVE	QUÉ HACE
acquire()	void	Coge un permiso. Si no queda ninguno, el hilo espera.
release()	void	Devuelve un permiso y deja pasar a un hilo que esperaba en acquire().
tryAcquire()	boolean	Intenta coger un permiso sin esperar: devuelve true si lo consigue y false si no.
availablePermits()	int	Indica cuántos permisos quedan libres.

Ejemplo de semáforo

Cinco hilos quieren usar un recurso que solo admite dos usuarios a la vez:

```
import java.util.concurrent.Semaphore;

public class SemaforoEjemplo {

    // Semáforo con 2 permisos: solo 2 hilos pueden acceder al recurso a la vez
    private static Semaphore semaforo = new Semaphore(2);

    public static void main(String[] args) {
        // Crear e iniciar 5 hilos que intentarán acceder al recurso limitado
        for (int i = 1; i <= 5; i++) {
            new Thread(new Tarea(i)).start();
        }
    }

    // Clase que representa una tarea que intentará acceder al recurso
    static class Tarea implements Runnable {
        private int id;

        public Tarea(int id) {
            this.id = id;
        }

        @Override
        public void run() {
            try {
                System.out.println("Hilo " + id + " intenta acceder al recurso.");

                // Adquiere un permiso del semáforo para acceder al recurso
                semaforo.acquire();
                System.out.println("Hilo " + id + " ha adquirido acceso al recurso.");

                // Simulación del uso del recurso por un tiempo
                Thread.sleep(2000);

                // Libera el permiso para que otros hilos puedan acceder
                System.out.println("Hilo " + id + " libera el recurso.");
                semaforo.release();

            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }
}
```

Una posible salida es:

```
Hilo 1 intenta acceder al recurso.
Hilo 1 ha adquirido acceso al recurso.
Hilo 2 intenta acceder al recurso.
Hilo 2 ha adquirido acceso al recurso.
Hilo 3 intenta acceder al recurso.
```

```
Hilo 5 intenta acceder al recurso.
Hilo 4 intenta acceder al recurso.
Hilo 1 libera el recurso.
Hilo 2 libera el recurso.
Hilo 5 ha adquirido acceso al recurso.
Hilo 3 ha adquirido acceso al recurso.
...
```

Explicación del ejemplo

- **new Semaphore(2)**: el semáforo se crea con 2 permisos, así que solo dos hilos pueden usar el recurso a la vez.
- **semaforo.acquire()**: cada hilo pide un permiso. Los dos primeros lo consiguen; los demás se quedan esperando.
- **semaforo.release()**: tras usar el recurso durante 2 segundos (simulado con sleep), el hilo devuelve su permiso y uno de los que esperaban puede entrar.
- Fíjate en que el orden en que entran los hilos que esperaban no tiene por qué ser el orden en que llegaron.

5.2.5 Monitores

Un **monitor** es una estructura de sincronización que garantiza la **exclusión mutua**: solo un hilo puede estar ejecutando un método dentro de él a la vez.

- **Implementación en Java**: los monitores se implementan fácilmente con la palabra clave **synchronized**. **Cada objeto de Java tiene un monitor**, que funciona como un cerrojo interno. Un **método synchronized** bloquea el propio objeto (this), y un **bloque synchronized (obj)** bloquea el objeto obj. Además, el monitor trae de serie wait() y notify().
- **Simplifica la sincronización**: el programador declara qué secciones son críticas sin preocuparse de cómo lo implementa el sistema operativo.

Formas de utilizar monitores en Java

- **Métodos sincronizados**: declarar un método como synchronized asegura que solo un hilo pueda ejecutarlo a la vez sobre el mismo objeto.

```
public synchronized void incrementar() {
    cuenta++;
}
```

- **Bloques de código sincronizados**: sincronizan solo una parte del método, en lugar del método entero. Así, el resto del método puede ejecutarse en paralelo.

```
public void incrementar() {
    // código que no necesita protección
    synchronized (this) {
        cuenta++;
    }
}
```

Ejemplo de monitor

Dos hilos incrementan cinco veces cada uno un contador compartido:

```java
public class MonitorEjemplo {

    // Clase Contador con métodos sincronizados
    static class Contador {
        private int cuenta = 0;

        // Método sincronizado para incrementar el contador
        public synchronized void incrementar() {
            cuenta++;
            System.out.println(Thread.currentThread().getName()
                    + " ha incrementado el contador a " + cuenta);
        }

        // Método para obtener el valor actual del contador
        public synchronized int getCuenta() {
            return cuenta;
        }
    }

    // Clase que representa una tarea que incrementa el contador
    static class IncrementarTarea implements Runnable {
        private Contador contador;

        public IncrementarTarea(Contador contador) {
            this.contador = contador;
        }

        @Override
        public void run() {
            for (int i = 0; i < 5; i++) {
                contador.incrementar();   // ya está sincronizado por dentro
                try {
                    Thread.sleep(100); // Simula trabajo y permite cambio de contexto
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }
        }
    }

    public static void main(String[] args) {
        Contador contador = new Contador();
        // Crear e iniciar dos hilos que incrementan el contador
        Thread hilo1 = new Thread(new IncrementarTarea(contador), "Hilo 1");
        Thread hilo2 = new Thread(new IncrementarTarea(contador), "Hilo 2");

        hilo1.start();
        hilo2.start();

        try {
            hilo1.join();
            hilo2.join();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }

        System.out.println("Valor final del contador: " + contador.getCuenta());
    }
}
```

Explicación del ejemplo

- **Método sincronizado incrementar()**: como es synchronized, solo un hilo puede ejecutarlo a la vez sobre el mismo objeto Contador. Así se evita la condición de carrera en cuenta++.
- **getCuenta()** también es synchronized: así, quien lea el contador ve siempre el último valor (recuerda que synchronized también garantiza la visibilidad).

- En **IncrementarTarea** no hace falta sincronizar nada: la protección ya está dentro del método incrementar(). Esta es una ventaja de los monitores: la sincronización queda encapsulada dentro de la clase que tiene los datos.
- El Thread.sleep(100) simula trabajo y hace que los hilos se vayan alternando. Al final, tras los join(), el valor es siempre **10**.

5.2.6 Condiciones de espera y notificación

Este apartado es un repaso del 4.1: **wait() y notify() forman parte del monitor**. Permiten que un hilo que está dentro de una sección crítica espere a que se cumpla una condición:

1. **wait()**: suspende un hilo dentro de la sección crítica hasta que reciba una notificación de otro hilo.
2. **notify()**: despierta a uno de los hilos que esperan en ese objeto (notifyAll() despierta a todos). El hilo despertado vuelve a coger el cerrojo y comprueba otra vez la condición en su while.
3. **Uso en bloques sincronizados**: deben usarse dentro de synchronized sobre el mismo objeto; si no, Java lanza una IllegalMonitorStateException. Además, wait() suelta el cerrojo mientras espera, para que otro hilo pueda entrar y cambiar la condición.

Ejemplo de condiciones de espera y notificación

```
public class CondicionesEsperaNotificacion {

    private static final Object monitor = new Object();
    private static boolean datoListo = false;

    public static void main(String[] args) {
        // Crear el hilo Consumidor
        Thread consumidor = new Thread(() -> {
            synchronized (monitor) {
                while (!datoListo) {
                    try {
                        System.out.println("Consumidor espera a que el dato esté listo...");
                        monitor.wait(); // Espera hasta recibir una notificación
                    } catch (InterruptedException e) {
                        Thread.currentThread().interrupt();
                        return;
                    }
                }
                System.out.println("Consumidor ha recibido el dato y está procesándolo.");
            }
        });

        // Crear el hilo Productor
        Thread productor = new Thread(() -> {
            synchronized (monitor) {
                System.out.println("Productor está generando el dato...");
                datoListo = true;
                monitor.notify(); // Notifica al consumidor de que el dato está listo
                System.out.println("Productor ha notificado al consumidor.");
            }
        });

        consumidor.start();
        productor.start();
    }
}
```

Explicación del ejemplo

- **Objeto monitor**: los dos hilos se sincronizan sobre el mismo objeto monitor, y la condición es la variable datoListo.
- **Hilo consumidor**: si el dato no está listo, espera con monitor.wait(), soltando el cerrojo.
- **Hilo productor**: entra en el monitor, pone datoListo a true y avisa con monitor.notify(). El consumidor se despierta, vuelve a comprobar la condición del while y, como ya se cumple, procesa el dato.
- **Si el productor llega primero**: pone datoListo a true y hace un notify() que no despierta a nadie. Cuando llega el consumidor, ve que el dato ya está listo y no llega a esperar. Esta es otra razón para usar un while con una condición, y no un wait() a secas: si el consumidor esperara sin comprobar nada, se quedaría esperando para siempre un aviso que ya pasó.

5.3 Mecanismos de sincronización avanzados

Java proporciona varias clases en el paquete java.util.concurrent para manejar situaciones de sincronización más avanzadas: controlar el acceso a recursos compartidos, coordinar el intercambio de datos o asegurar que varios hilos alcancen un estado común antes de continuar.

5.3.1 Clase Semaphore

La clase Semaphore, que controla el acceso a un recurso con un número limitado de permisos, ya se ha estudiado en el apartado 5.2.4, junto con sus métodos y un ejemplo completo.

5.3.2 Clase Exchanger

- **Intercambio sincronizado**: permite sincronizar **dos hilos** e intercambiar objetos entre ellos. Cada hilo llega al punto de intercambio con un objeto, espera a que llegue el otro y, cuando están los dos, se los intercambian.
- **Métodos principales**: **exchange(V x)** entrega x y devuelve el objeto del otro hilo; **exchange(V x, long timeout, TimeUnit unit)** hace lo mismo, pero si el otro hilo no llega en ese tiempo lanza una TimeoutException.



Ejemplo de uso

Un productor genera datos y los intercambia con un consumidor, que le devuelve un acuse de recibo:

```
import java.util.concurrent.Exchanger;

public class Intercambio {

    public static void main(String[] args) {
        Exchanger<String> exchanger = new Exchanger<>();
```

```
        Thread productor = new Thread(() -> {
            try {
                String recibido = exchanger.exchange("Datos generados");
                System.out.println("Productor recibe: " + recibido);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        Thread consumidor = new Thread(() -> {
            try {
                String recibido = exchanger.exchange("Recibido, gracias");
                System.out.println("Consumidor recibe: " + recibido);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        productor.start();
        consumidor.start();
    }
}
```

La salida es:

```
Productor recibe: Recibido, gracias
Consumidor recibe: Datos generados
```

Cada hilo entrega su cadena y recibe la del otro. Si uno de los dos llega antes al exchange(), espera allí hasta que llegue el otro. El orden de las dos líneas puede variar.

5.3.3 Clase CountDownLatch

- **Descripción**: permite que uno o varios hilos esperen hasta que otro conjunto de hilos haya completado sus tareas.
- **Funcionamiento**: se crea con un contador, por ejemplo new CountDownLatch(3). Cada llamada a countDown() le resta 1 y, cuando llega a 0, el 'pestillo' (*latch*) se abre y los hilos que esperaban en await() continúan. Es **de un solo uso**: una vez abierto, no se puede volver a cerrar. Esto es lo que lo diferencia de CyclicBarrier.
- **Métodos principales**: **await()** bloquea el hilo hasta que el contador llega a cero; **countDown()** decrementa el contador en uno.

Ejemplo de uso

Un programa principal espera a que tres servicios terminen de inicializarse antes de continuar:

```
import java.util.concurrent.CountDownLatch;

public class ServiceInitialization {
    private static final int NUM_SERVICES = 3;
    private static final CountDownLatch latch = new CountDownLatch(NUM_SERVICES);

    static class Service implements Runnable {
        @Override
        public void run() {
            try {
                System.out.println(Thread.currentThread().getName() + " inicializando...");
                Thread.sleep(1000); // Simulación de trabajo
                System.out.println(Thread.currentThread().getName() + " listo.");
            } catch (InterruptedException e) {
```

```
                Thread.currentThread().interrupt();
            } finally {
                latch.countDown(); // Servicio completo. Se avisa siempre, aunque falle
            }
        }
    }

    public static void main(String[] args) {
        for (int i = 0; i < NUM_SERVICES; i++) {
            new Thread(new Service()).start();
        }

        try {
            latch.await(); // Espera a que todos los servicios estén listos
            System.out.println("Todos los servicios están inicializados. "
                    + "Iniciando aplicación principal.");
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

Explicación del ejemplo

- El latch se crea con el número de servicios, 3.
- Cada servicio simula su inicialización y, al terminar, llama a countDown(). Se coloca en un bloque **finally** para que se descuente **siempre**, aunque el servicio falle; si no, el hilo main podría quedarse esperando para siempre.
- El hilo main espera en latch.await() y solo continúa cuando los tres servicios han avisado.

5.3.4 Clase CyclicBarrier

La clase CyclicBarrier hace que varios hilos **se esperen unos a otros** hasta que todos alcancen un punto común, llamado **barrera**.

- **Sincronización repetida**: permite que varios hilos se sincronicen repetidamente en la barrera.
- **Método await()**: hace que el hilo actual espere hasta que todos los hilos han llegado a la barrera.
- **Reutilizable**: cuando pasan todos, la barrera se reinicia sola y sirve para el siguiente paso; por eso es 'cíclica'. Al crearla se le puede dar una **tarea que se ejecuta cada vez que se abre**. El método reset() solo se usa para forzar el reinicio en caso de error.

Ejemplo de uso

Una aplicación realiza una serie de pasos en paralelo y, en cada paso, necesita que todos los hilos terminen antes de pasar al siguiente:

```
import java.util.concurrent.BrokenBarrierException;
import java.util.concurrent.CyclicBarrier;


public class TareasPorPasos {


    private static final int HILOS = 3;
    private static final CyclicBarrier barrera = new CyclicBarrier(HILOS,
            () -> System.out.println("--- Todos han llegado. Siguiente paso ---"));
```

```
    public static void main(String[] args) {
        for (int i = 1; i <= HILOS; i++) {
            new Thread(() -> {
                try {
                    for (int paso = 1; paso <= 3; paso++) {
                        System.out.println(Thread.currentThread().getName()
                                + " termina el paso " + paso);
                        barrera.await();   // espera a los demás; se reinicia sola
                    }
                } catch (InterruptedException | BrokenBarrierException e) {
                    Thread.currentThread().interrupt();
                }
            }, "Hilo-" + i).start();
        }
    }
}
```

Una posible salida es:

```
Hilo-1 termina el paso 1
Hilo-2 termina el paso 1
Hilo-3 termina el paso 1
--- Todos han llegado. Siguiente paso ---
Hilo-3 termina el paso 2
Hilo-1 termina el paso 2
Hilo-2 termina el paso 2
--- Todos han llegado. Siguiente paso ---
...
```

Aunque el orden de los hilos dentro de cada paso cambia, **ningún hilo empieza el paso 2 hasta que los tres han terminado el paso 1**. La misma barrera sirve para los tres pasos.

	COUNTDOWNLATCH	CYCLICBARRIER
Quién espera	Uno o varios hilos esperan a que otros terminen.	Los propios hilos se esperan entre sí.
Cómo se abre	Cuando el contador llega a 0 con countDown().	Cuando han llegado todos los hilos con await().
Reutilización	De un solo uso.	Se reinicia sola después de cada apertura.

5.4 Buenas prácticas y estrategias para la sincronización de hilos

La sincronización es imprescindible para evitar los problemas de concurrencia, pero **su uso excesivo reduce el rendimiento** y puede provocar problemas de bloqueo. Hay que encontrar un equilibrio y aplicar estrategias eficientes.

5.4.1 Minimizar el uso de sincronización

- **Reducir la sobrecarga**: la sincronización añade trabajo extra a la ejecución (los hilos se paran y esperan), así que conviene usar solo la imprescindible.
- **Evitar bloqueos**: cuanta menos sincronización, menos riesgo de interbloqueos e inanición.
- **Mejorar la escalabilidad**: con menos sincronización, el programa aprovecha mejor los núcleos y soporta más carga de trabajo.

Estrategias para minimizar la sincronización

- **Evitar la sincronización innecesaria**: reducir las secciones críticas al mínimo. Si un recurso no es compartido, no necesita sincronización.
- **Usar estructuras de datos seguras para hilos**: Java ofrece colecciones concurrentes, como **ConcurrentHashMap** y **CopyOnWriteArrayList**, que ya se sincronizan por dentro.
- **Sincronización de grano fino**: dividir las secciones sincronizadas en partes más pequeñas (por ejemplo, un bloque synchronized en lugar de un método entero) para permitir más paralelismo.

5.4.2 Estrategias para evitar el interbloqueo

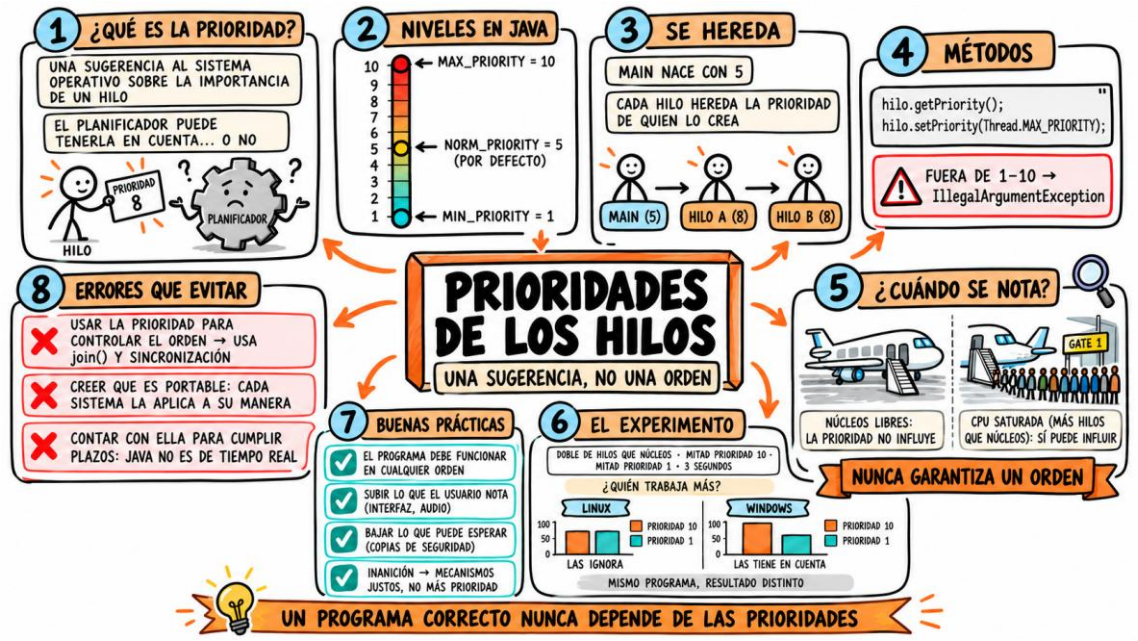
1. **Orden predefinido**: establecer un orden estricto para adquirir los bloqueos, de forma que todos los hilos pidan los recursos en el mismo orden. En el ejemplo Interbloqueo del apartado 5.1, si los dos hilos pidieran primero el recurso 1 y después el 2, el problema desaparecería.
2. **No esperar con recursos cogidos**: no esperar a otros hilos (await(), join(), acquire()…) mientras se tiene un recurso que ellos necesitan. Primero se suelta y después se espera. Es exactamente el error que corregimos en el ejemplo integrador del apartado 3.2.2.
3. **Minimizar el tiempo de bloqueo**: mantener los bloqueos solo el tiempo estrictamente necesario, para que los demás hilos puedan acceder antes a los recursos.
4. **Usar temporizadores**: utilizar métodos con tiempo de espera, como **tryLock()**, para que un hilo no espere indefinidamente y pueda abandonar el intento.

5.4.3 Estrategias para evitar la inanición

- **Prioridades equilibradas**: no confiar en las prioridades para repartir los recursos. En Java son solo una sugerencia al sistema operativo y no garantizan un reparto justo.
- **Mecanismos de sincronización justos**: usar el **modo justo** (*fair mode*) de mecanismos como ReentrantLock (new ReentrantLock(true)), que concede el acceso en el orden en que se pidió.
- **Retroceso y reintento**: si un hilo no consigue el recurso, espera un **tiempo aleatorio** antes de volver a intentarlo. Es también la solución al **bloqueo activo**: así las dos personas del pasillo dejan de apartarse a la vez.
- **Semáforos justos**: crear el semáforo en modo justo, new Semaphore(2, true), para que los permisos se concedan por orden de llegada, como en una cola.

6 Prioridades de los hilos

Ya hemos visto que cada hilo tiene una prioridad y que es solo una sugerencia. En este apartado vamos a estudiar con detalle qué es la prioridad, cómo se maneja en Java, qué efecto tiene realmente durante la ejecución y cuándo merece la pena utilizarla.



6.1 Concepto de prioridad en los hilos

La **prioridad** es una **sugerencia** que damos al sistema operativo sobre la importancia de cada hilo. El planificador puede tenerla en cuenta para decidir qué hilo se ejecuta cuando **hay más hilos listos que núcleos libres**, pero **no está obligado** a hacerlo.

Es importante entender bien esta idea: una prioridad más alta no significa que el hilo se ejecute antes ni que termine antes. Solo significa que, si hay que elegir, el sistema operativo **puede** darle preferencia.

6.2 Niveles de prioridad

En Java, la prioridad de un hilo es un número entero entre 1 y 10. La clase Thread define tres constantes para los valores más usados:

CONSTANTE	VALOR	SIGNIFICADO
Thread.MIN_PRIORITY	1	Prioridad mínima.
Thread.NORM_PRIORITY	5	Prioridad normal. Es la prioridad por defecto.
Thread.MAX_PRIORITY	10	Prioridad máxima.

El hilo main arranca con prioridad 5. **Cada hilo nuevo hereda la prioridad del hilo que lo crea**: si un hilo de prioridad 8 crea otro hilo, el nuevo nace también con prioridad 8.

6.3 Establecimiento y modificación de prioridades

La clase Thread ofrece dos métodos para trabajar con la prioridad:

- **getPriority()**: devuelve la prioridad actual del hilo.
- **setPriority(int priority)**: establece una nueva prioridad, con un valor entre 1 y 10. Con un valor fuera de ese rango (0 u 11, por ejemplo), lanza una IllegalArgumentException. Lo normal es llamarlo antes de start().

Ejemplo básico

```
public class PrioridadHilos {
    public static void main(String[] args) {
        Thread hilo1 = new Thread(() -> System.out.println("Hilo 1 ejecutándose"));
        Thread hilo2 = new Thread(() -> System.out.println("Hilo 2 ejecutándose"));


        // Establecer prioridades
        hilo1.setPriority(Thread.MIN_PRIORITY); // Prioridad baja
        hilo2.setPriority(Thread.MAX_PRIORITY); // Prioridad alta


        // Iniciar los hilos
        hilo1.start();
        hilo2.start();
    }
}
```

Aunque hilo2 tiene la prioridad máxima y hilo1 la mínima, **los mensajes pueden salir en cualquier orden**. Hay núcleos libres de sobra para los dos hilos, así que la prioridad no influye: el orden depende de cuál arranque antes.

6.4 Efectos de las prioridades durante la ejecución

- **Las prioridades no son estrictas**: los hilos con mayor prioridad tienen más probabilidad de ejecutarse antes, pero el sistema operativo puede ignorar las prioridades según su planificador. Además, **la prioridad solo se nota cuando la CPU está saturada**, es decir, cuando hay más hilos listos que núcleos. Si sobran núcleos, cada hilo tiene el suyo y la prioridad no influye.
- **La ejecución no está garantizada**: en un programa con hilos de distintas prioridades, los de mayor prioridad tienden a completar sus tareas antes, pero no siempre. Aunque la CPU esté saturada, la prioridad solo hace **más probable** que un hilo reciba más tiempo de CPU. **Nunca garantiza un orden.**

Ejemplo práctico

Para ver el efecto de las prioridades hay que saturar la CPU. El siguiente programa lanza el **doble de hilos que núcleos**, la mitad con prioridad 10 y la otra mitad con prioridad 1, y los pone a trabajar durante 3 segundos contando vueltas. Al final suma el trabajo de cada grupo. ¿Quién habrá trabajado más?

```
public class CarreraPrioridades {


    public static void main(String[] args) throws InterruptedException {
        int nucleos = Runtime.getRuntime().availableProcessors();
        int numHilos = nucleos * 2;          // más hilos que núcleos: CPU saturada
        long[] vueltas = new long[numHilos];
        Thread[] hilos = new Thread[numHilos];


        for (int i = 0; i < numHilos; i++) {
            int id = i;
            hilos[i] = new Thread(() -> {
                long fin = System.currentTimeMillis() + 3000;   // trabaja 3 segundos
                long contador = 0;
```

```
                while (System.currentTimeMillis() < fin) {
                    contador++;
                }
                vueltas[id] = contador;
            });
            // la mitad con prioridad máxima y la otra mitad con mínima
            hilos[i].setPriority(i % 2 == 0 ? Thread.MAX_PRIORITY : Thread.MIN_PRIORITY);
        }


        for (Thread h : hilos) h.start();
        for (Thread h : hilos) h.join();


        long alta = 0, baja = 0;
        for (int i = 0; i < numHilos; i++) {
            if (i % 2 == 0) alta += vueltas[i]; else baja += vueltas[i];
        }
        System.out.println("Núcleos: " + nucleos + "   ·  Hilos: " + numHilos);
        System.out.printf("Trabajo de los hilos con prioridad 10: %,d vueltas%n", alta);
        System.out.printf("Trabajo de los hilos con prioridad 1 : %,d vueltas%n", baja);
    }
}
```

Explicación del ejemplo

- Runtime.getRuntime().availableProcessors() devuelve el número de núcleos lógicos del equipo. Lanzando el doble de hilos, nos aseguramos de que la CPU esté saturada.
- Cada hilo cuenta vueltas durante 3 segundos y guarda el resultado en su posición del array vueltas. Como cada hilo escribe en una posición distinta y el main solo lee el array después de los join(), no hay condición de carrera.
- Los hilos de posición par tienen prioridad 10 y los de posición impar, prioridad 1.
- **El resultado depende del sistema operativo**: en Windows lo normal es que los hilos de prioridad 10 acumulen bastante más trabajo que los de prioridad 1. En Linux, en cambio, los dos grupos suelen quedar prácticamente igualados, porque por defecto Java no aplica allí las prioridades. Si cierras otros programas o cambias el número de hilos, también cambiarán las cifras.

6.5 Buenas prácticas en el uso de prioridades

- **Evitar dependencias estrictas**: las prioridades no garantizan el orden de ejecución, así que los programas deben funcionar correctamente sea cual sea el orden de los hilos. Para controlar el orden se usan join() y los mecanismos de sincronización.
- **Usar prioridades solo cuando sea necesario**: subir un poco la prioridad de lo que el usuario nota (la interfaz, la reproducción de audio) y bajar la de lo que puede esperar (copias de seguridad, indexado de archivos). Pero nunca contar con ello para cumplir un plazo: Java estándar **no es un sistema de tiempo real**.
- **Considerar la portabilidad**: cada sistema operativo traduce las prioridades de Java a su manera. Windows agrupa los 10 niveles de Java en 7, y Linux, por defecto, las ignora. El mismo programa se comporta de forma distinta según dónde se ejecute. Usar las constantes (MIN_PRIORITY, NORM_PRIORITY, MAX_PRIORITY) hace el código más legible, pero **no lo hace portable**.
- **Evitar problemas de inanición**: ajustar las prioridades de manera razonable para que los hilos de prioridad baja no se queden sin ejecutar. Y si un hilo no consigue

un recurso, la solución no es subirle la prioridad, sino usar **mecanismos justos** (apartado 5.4.3).

Conclusión

- La prioridad es una **sugerencia** al sistema operativo: solo se nota con la CPU saturada, y cada sistema la aplica a su manera.
- Un programa correcto **nunca depende de las prioridades**. Para controlar el orden se usan join() y la sincronización; para un reparto justo, los mecanismos justos.
- Úsalas **con moderación**, para dar preferencia a lo que el usuario nota, y deja el resto en NORM_PRIORITY.

7 Bibliografía

1. Cortijo Bon, Carlos A. (2021). *Programación de servicios y procesos*. Editorial Síntesis (España). ISBN: 978-84-1357-104-1.
2. Córcoles Tendero, José Eduardo (2014). *Programación de servicios y procesos*. Editorial RA-MA (España). ISBN: 978-84-9964-391-5.
3. Paniagua Martín, Fernando (2022). *Programación de servicios y procesos*. Editorial Paraninfo. ISBN: 978-84-1366-526-9.